INTRODUCTION TO DEEP LEARNING

Course number: 00240332

Lecture 9: Recurrent Neural Networks

Xiaolin Hu (胡晓林)

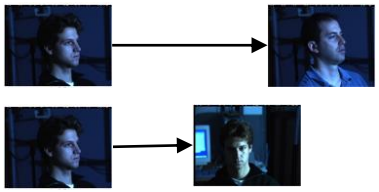
Dept. of Computer Science and Technology

Tsinghua University

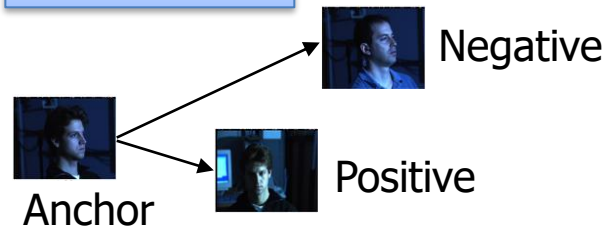
Last lecture review

I. Image classification

Contrastive loss

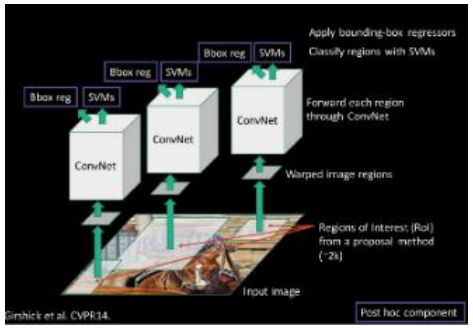


Triplet loss

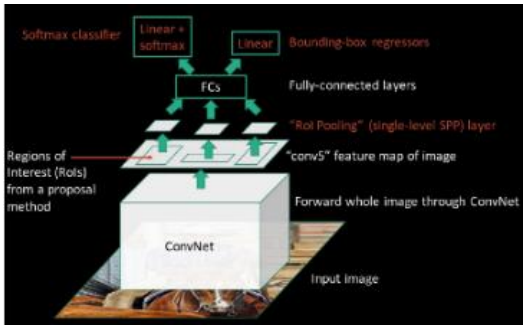


II. Object detection

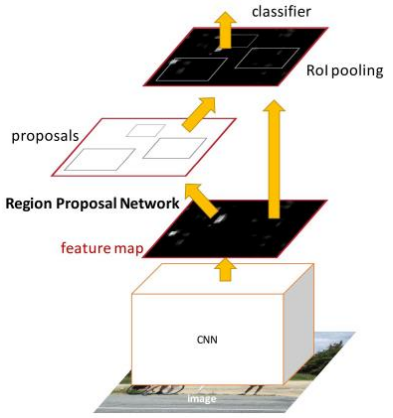
R-CNN



Fast R-CNN



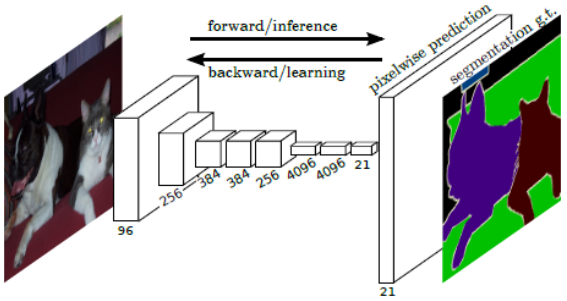
Faster R-CNN



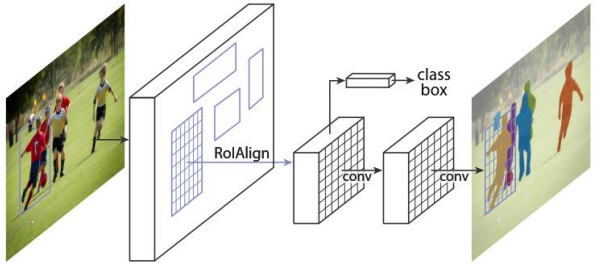
Last lecture review

III. Image segmentation

FCN

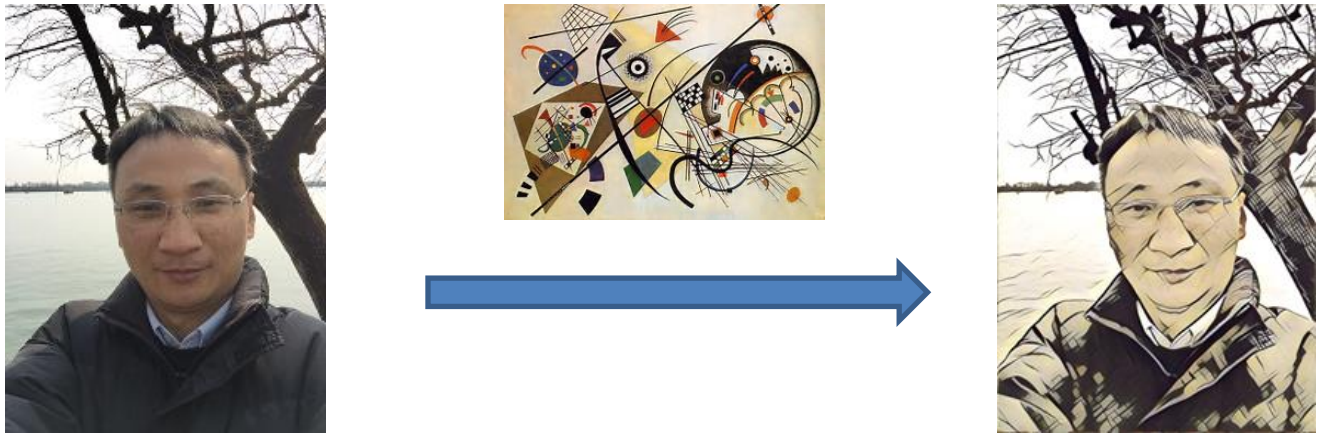


Mask R-CNN



Recommended reading: Image style transformation

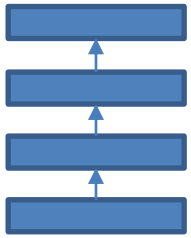
Gatys, Leon A., Alexander S. Ecker, and Matthias Bethge. "A neural algorithm of artistic style." CVPR 2016.



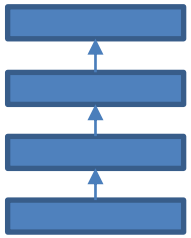
A Neural Algorithm of Artistic Style

Feature representation

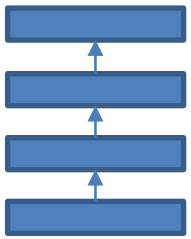
Style representation



$\phi^l(p)$ $\approx$ $\phi^l(x)$



$G^l(x)$ $\approx$ $G^l(s)$





p

noise

x

Final solution x^*





s

Content loss

Style loss

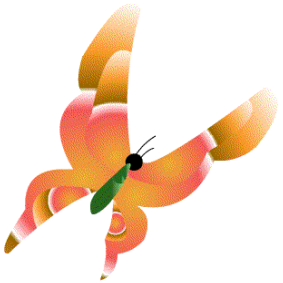
$$\min_{x} \sum_{l} \left(\alpha \left|\left| \phi^l(p) - \phi^l(x) \right|\right|_2^2 + \beta \left|\left| G^l(s) - G^l(x) \right|\right|_2^2 \right)$$

where $\alpha, \beta > 0$

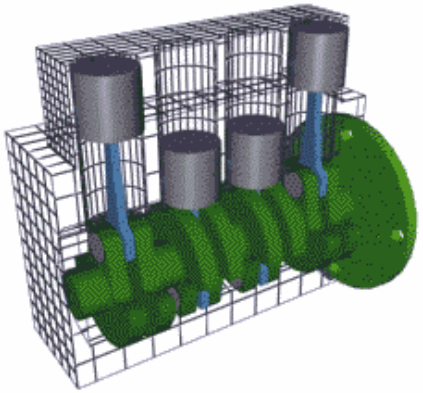
Outline

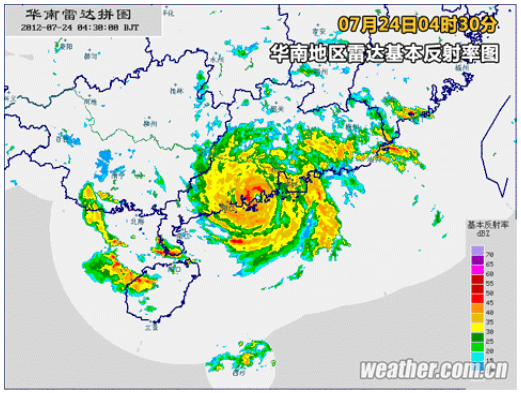
I. Simple RNNs
II. Gated RNNs
III. Applications to speech recognition
IV. Summary

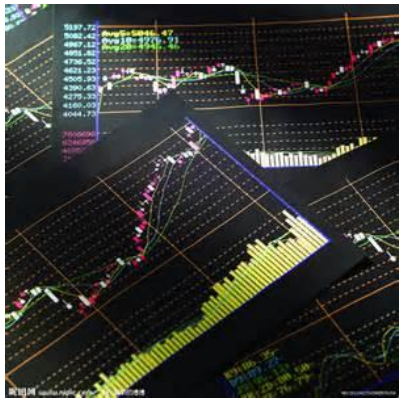
Dynamic systems





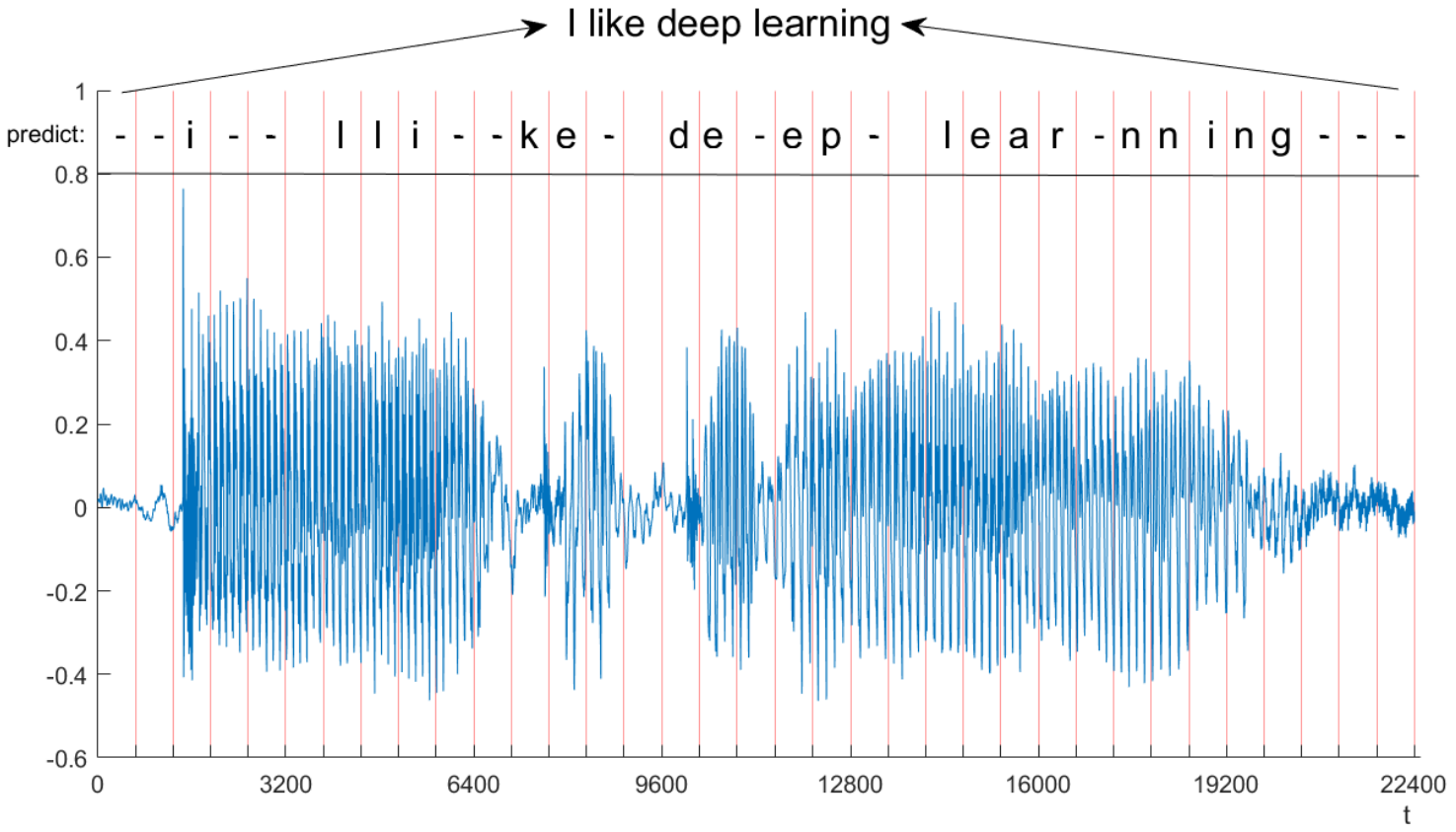






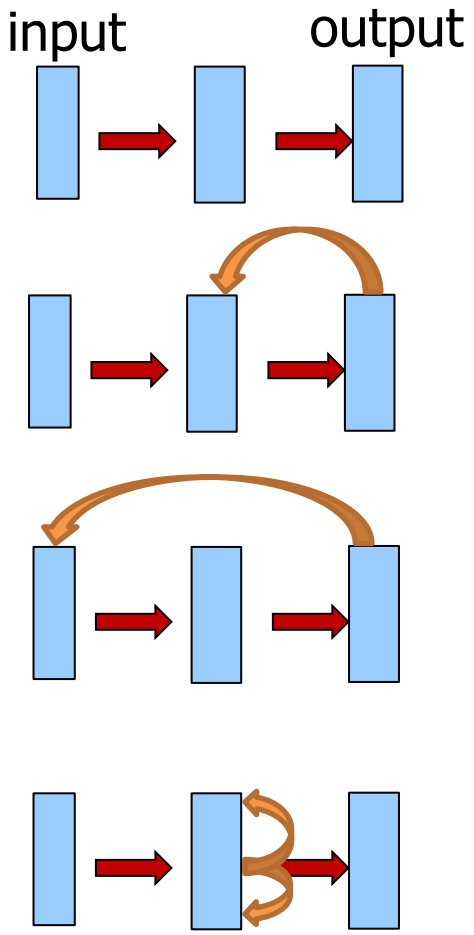


Speech recognition



Feedback connections

- Feedforward networks
  - No feedback
- Recurrent networks
  - Between-layer feedback
  - Within-layer feedback



With feedback connections, the state (and therefore output) of neurons will change over time

RNNs in general

- The states of the neurons in RNN evolve over time

$$\boldsymbol{h}(t) = f(\boldsymbol{h}(t-1), \boldsymbol{x}(t), \boldsymbol{y}(t))$$

  - $\boldsymbol{h}$ denotes the states of *all neurons*
  - $\boldsymbol{x}$ denotes input to the network
  - $\boldsymbol{y}$ denotes output of the network
  - f is a nonlinear function
- Often, the output neurons $\boldsymbol{y}$ are separated from the above equation

$$\boldsymbol{y}(t) = g(\boldsymbol{h}(t), \boldsymbol{x}(t))$$

  where g denotes the output function

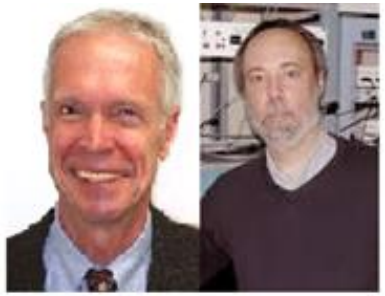
- Such systems are termed (discrete) dynamic systems

- Linear VS nonlinear: f is linear or nonlinear

History of RNN



Grossberg



Hopfield & Tank

Brain dynamics

Learning



Jordan



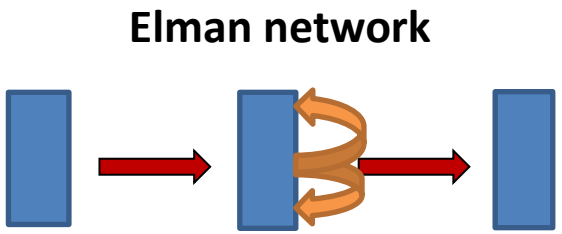
Elman



Schmidhuber

1957 1984 1990 1997

Elman network



Input $\boldsymbol{x}$ Hidden $\boldsymbol{h}$ Output $\boldsymbol{y}$



January 22, 1948 – June 28, 2018

Dynamic system

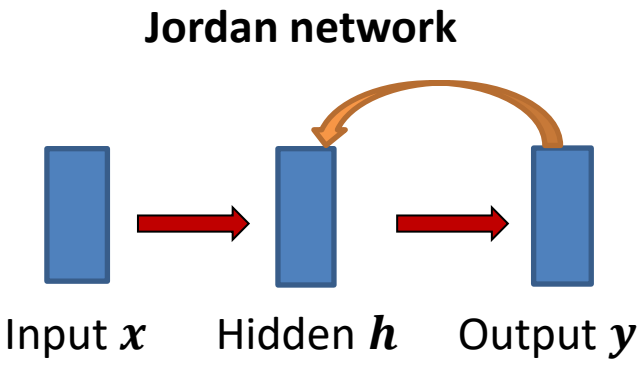
$$\boldsymbol{h}(t) = \sigma_h(\boldsymbol{W}_h\boldsymbol{x}(t) + \boldsymbol{U}_h\boldsymbol{h}(t-1) + \boldsymbol{b}_h)$$

$$\boldsymbol{y}(t) = \sigma_y(\boldsymbol{W}_y\boldsymbol{h}(t) + \boldsymbol{b}_y)$$

- $\boldsymbol{x}$: input; $\boldsymbol{y}$: output
- $\boldsymbol{h}$: hidden state
- σ_h and σ_y: activation functions
- $\boldsymbol{W}, \boldsymbol{U}, \boldsymbol{b}$: parameters

- Jeffrey Elman
  - BS in 1969 from Harvard University
  - Ph.D. in 1977 from the University of Texas at Austin
- Professor of cognitive science at the UCSD

Jordan network



Dynamic system

$$\boldsymbol{h}(t) = \sigma_h(\boldsymbol{W}_h\boldsymbol{x}(t) + \boldsymbol{U}_h\boldsymbol{y}(t-1) + \boldsymbol{b}_h)$$

$$\boldsymbol{y}(t) = \sigma_y(\boldsymbol{W}_y\boldsymbol{h}(t) + \boldsymbol{b}_y)$$

- $\boldsymbol{x}$: input; $\boldsymbol{y}$: output
- $\boldsymbol{h}$: hidden state
- σ_h and σ_y: activation functions
- $\boldsymbol{W}, \boldsymbol{U}, \boldsymbol{b}$: parameters



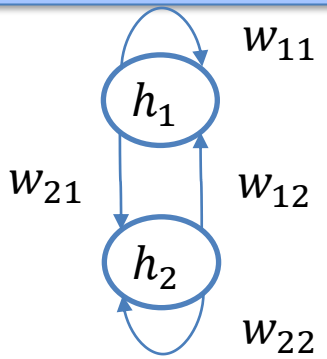
- Michael I Jordan
  - BS in Psychology in 1978 from the Louisiana State University,
  - MS in Mathematics in 1980 from Arizona State University
  - PhD in Cognitive Science in 1985 from the UCSD
- At UCSD, Jordan was a student of David Rumelhart
- Now at UC Berkeley

Back-propagation through time (BPTT)

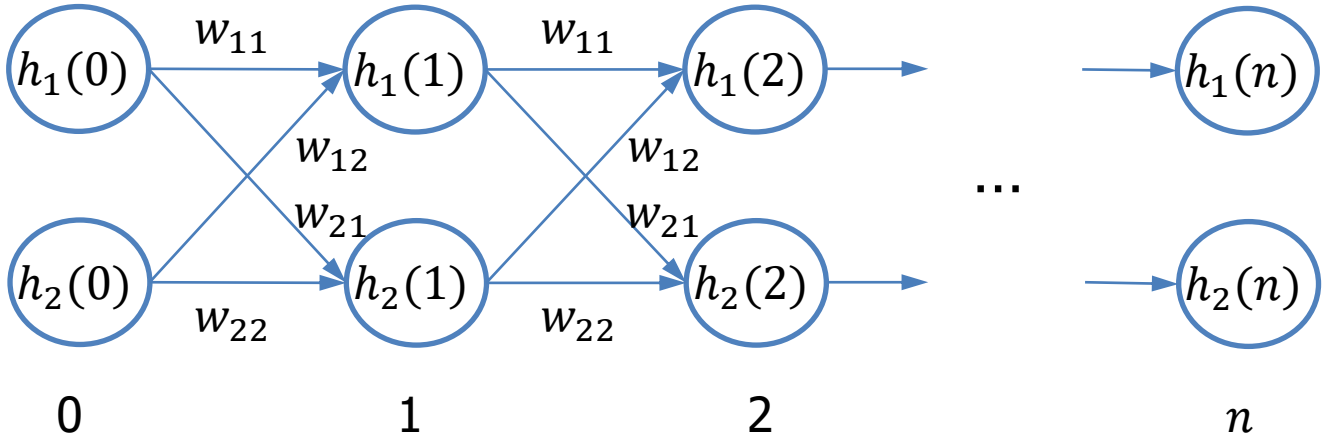
Unfold the temporal operation of the network into a layered feedforward network, the topology of which grows by one layer at every time step.

Consider a linear system without input:

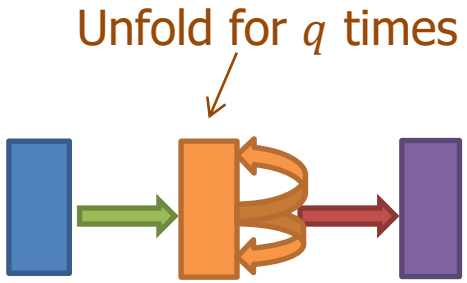
$$h_1(t+1) = w_{11}h_1(t) + w_{12}h_2(t)$$
$$h_2(t+1) = w_{21}h_1(t) + w_{22}h_2(t)$$



Unfold through time:

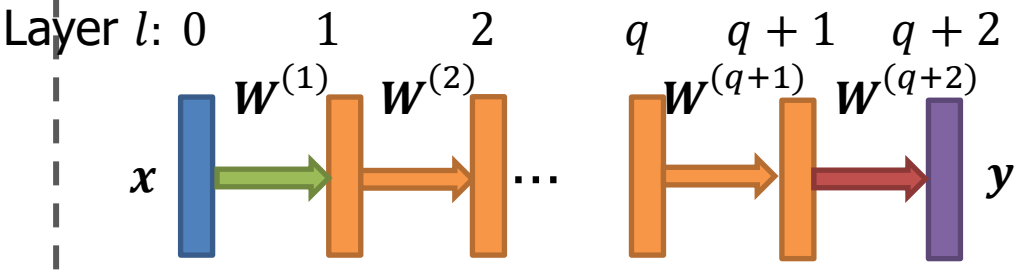


Unfold the Elman network



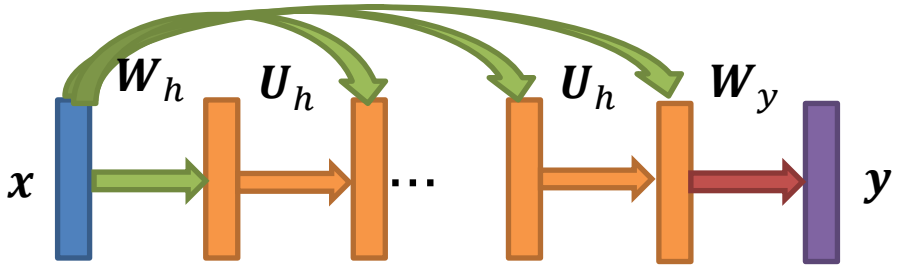
$$\boldsymbol{h}(t) = \sigma_h(\boldsymbol{W}_h\boldsymbol{x}(t) + \boldsymbol{U}_h\boldsymbol{h}(t-1) + \boldsymbol{b}_h)$$
$$\boldsymbol{y}(t) = \sigma_y(\boldsymbol{W}_y\boldsymbol{h}(t) + \boldsymbol{b}_y)$$

- Case 1:
  - $\boldsymbol{x}$ is only present at the first step
  - Label $\boldsymbol{r}$ is only present at the last step



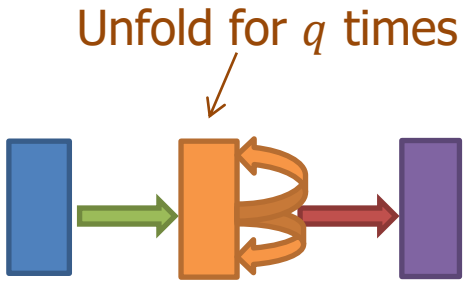
$$\boldsymbol{W}^{(1)} = \boldsymbol{W}_h, \boldsymbol{W}^{(2)} = \cdots = \boldsymbol{W}^{(q+1)} = \boldsymbol{U}_h$$
$$\boldsymbol{W}^{(q+2)} = \boldsymbol{W}_y$$

- Case 2:
  - $\boldsymbol{x}$ is fixed but present at all steps
  - Label $\boldsymbol{r}$ is only present at the last step
  - E.g., image classification (Liang, Hu, CVPR 2015)



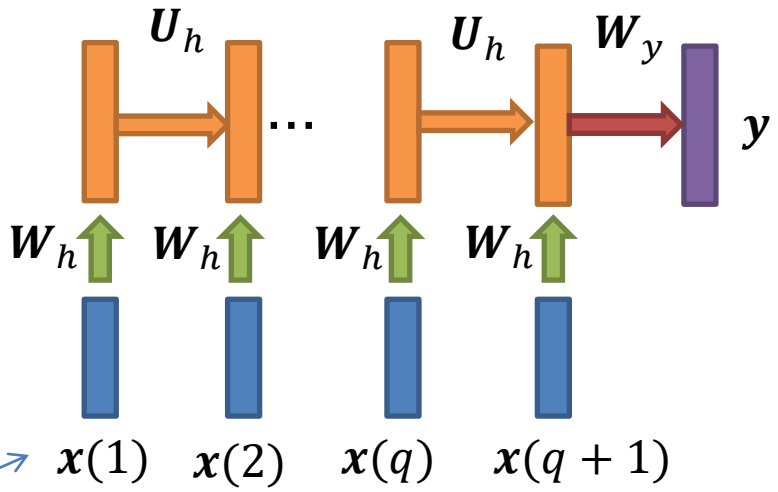
(Arrows in the same color share weights)

Unfold the Elman network



$$\boldsymbol{h}(t) = \sigma_h(\boldsymbol{W}_h\boldsymbol{x}(t) + \boldsymbol{U}_h\boldsymbol{h}(t-1) + \boldsymbol{b}_h)$$
$$\boldsymbol{y}(t) = \sigma_y(\boldsymbol{W}_y\boldsymbol{h}(t) + \boldsymbol{b}_y)$$

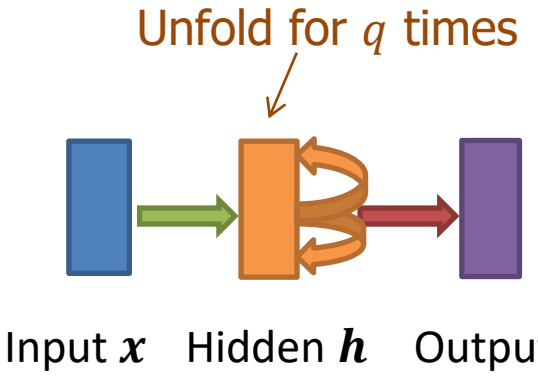
- Case 3:
  - $\boldsymbol{x}$ is time-varying
  - Label $\boldsymbol{r}$ is only present at the last step
  - E.g., sentence classification



This can be viewed as layer 0 attached to the orange backbone

(Arrows in the same color share weights)

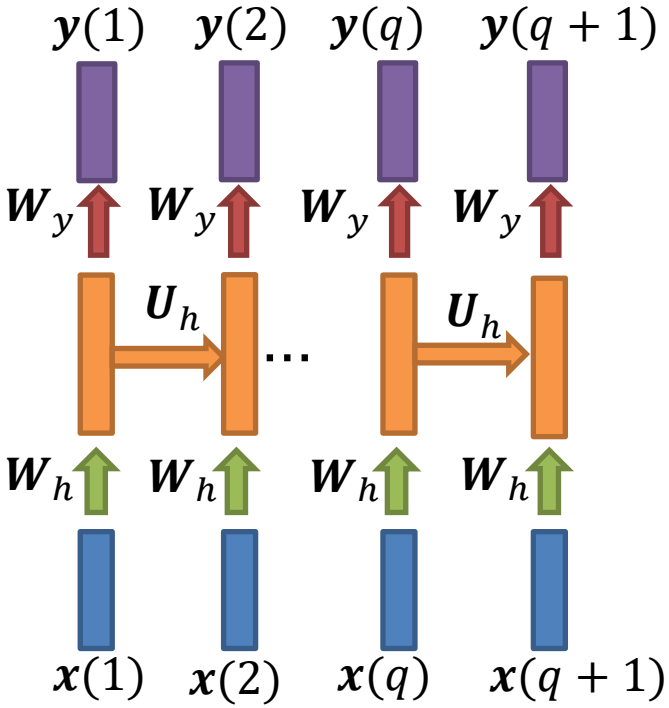
Unfold the Elman network



$$\boldsymbol{h}(t) = \sigma_h(\boldsymbol{W}_h\boldsymbol{x}(t) + \boldsymbol{U}_h\boldsymbol{h}(t-1) + \boldsymbol{b}_h)$$
$$\boldsymbol{y}(t) = \sigma_y(\boldsymbol{W}_y\boldsymbol{h}(t) + \boldsymbol{b}_y)$$

Do you know other cases?

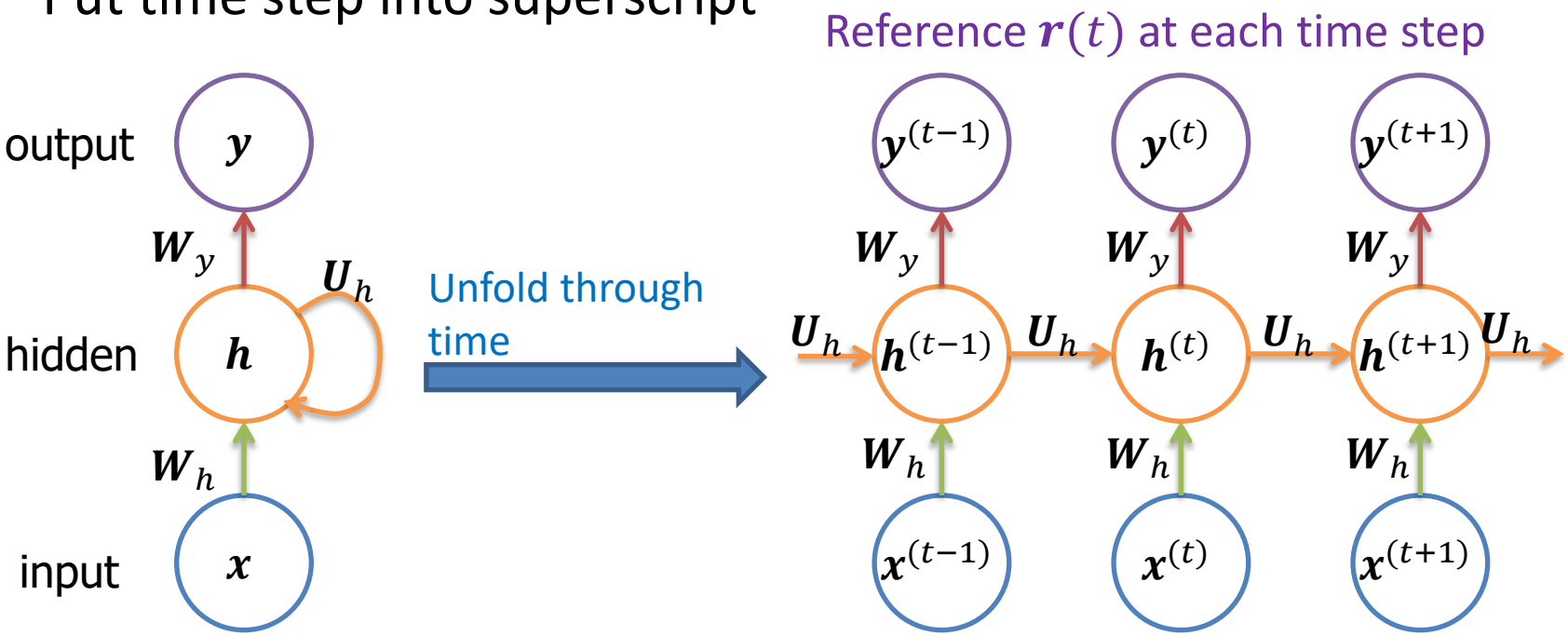
- Case 4:
  - $\boldsymbol{x}$ is time-varying
  - Label $\boldsymbol{r}$ is present at all steps
  - E.g., speech recognition



(Arrows in the same color share weights)

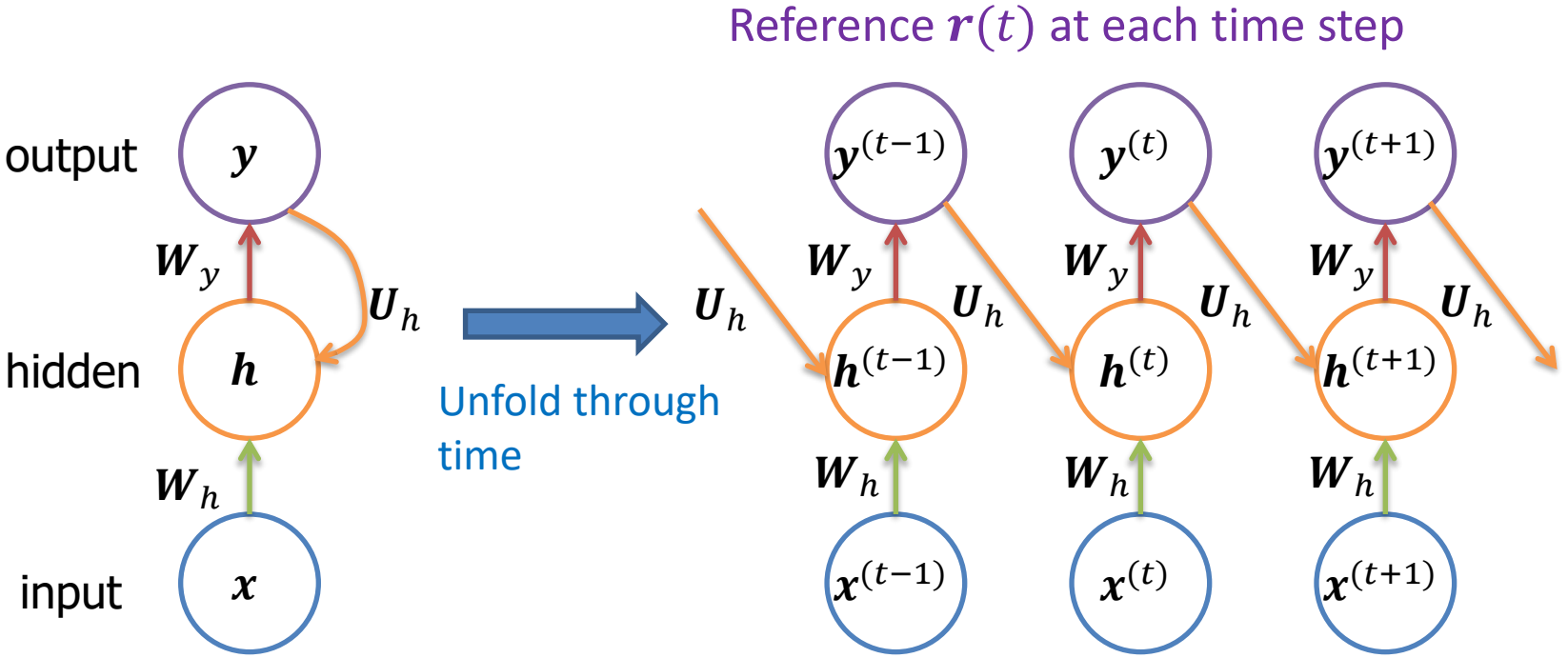
Simplified illustration (Elman network)

- Use circles to represent vectors (one circle one layer)
- Put time step into superscript



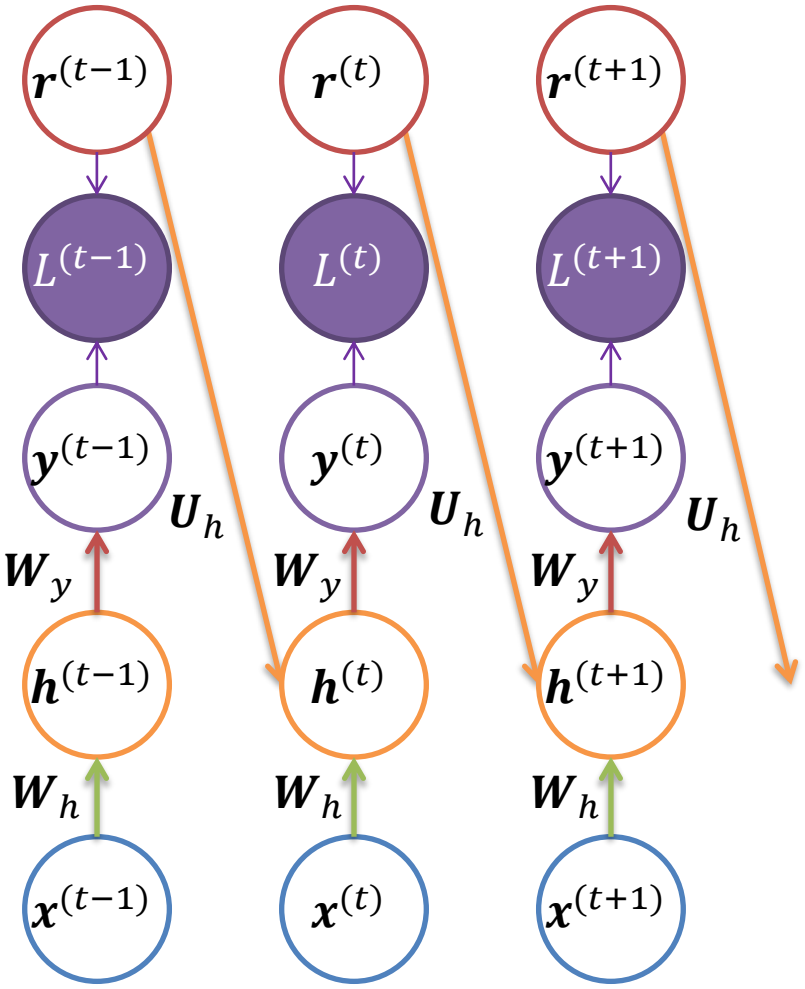
- The forward propagation runtime is $O(q)$ and cannot be reduced

Unfold the Jordan network



- During training, if the hidden states take the output of the previous step as input ← Free running training
- The forward propagation runtime is also $O(q)$

Teacher forcing



- The hidden states can also take the reference signal of the previous step as input

Teacher forcing training

- If the loss is based on comparing $\boldsymbol{y}(t)$ and $\boldsymbol{r}(t)$, all time steps are decoupled and training can be parallelized

Teacher forcing

- However, in testing, there is no reference signal and we have to use the network's output at time t
- The kind of inputs that the network sees during training could be quite different from the kind of inputs that it will see at test time



- To mitigate this problem:
  - Alternately use teacher-forced inputs and free-running inputs for a number of time steps
  - Randomly choose between the teacher-forced input and free-running input at every time step

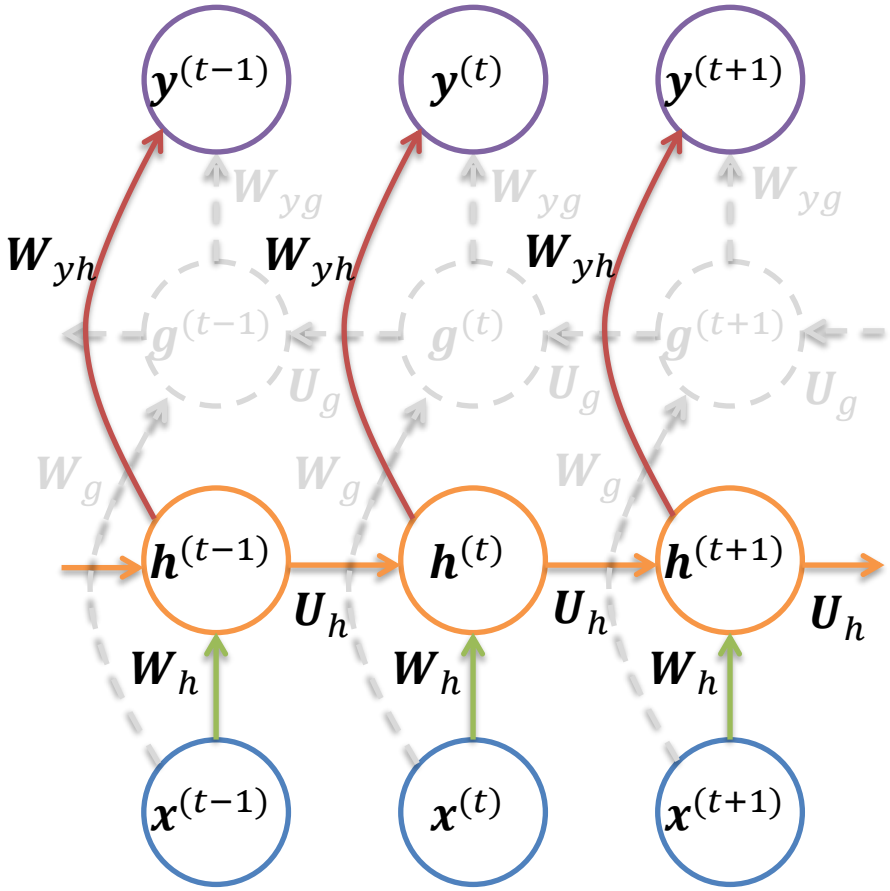
Bidirectional RNN

- In many applications, the prediction at the current time step may depend on the whole input sequence (both the past and the future)

> *Bank is the side of a river.*
> *Bank provides various financial services.*

- Bidirection RNNs combine an RNN that moves forward through time with another RNN that moves backward through time
- The output of the entire network at every time step then receives two inputs

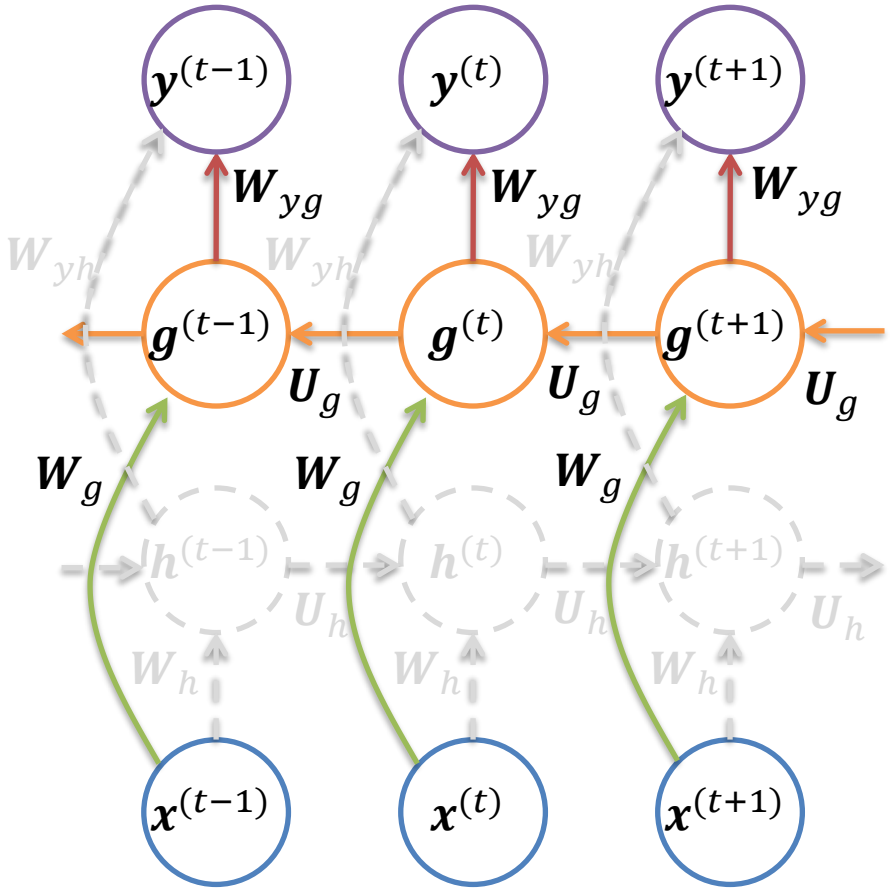
Bidirectional RNN



W_{yh}: connection weights from the sub-RNN g to output

One sub-RNN moves forward, same as before

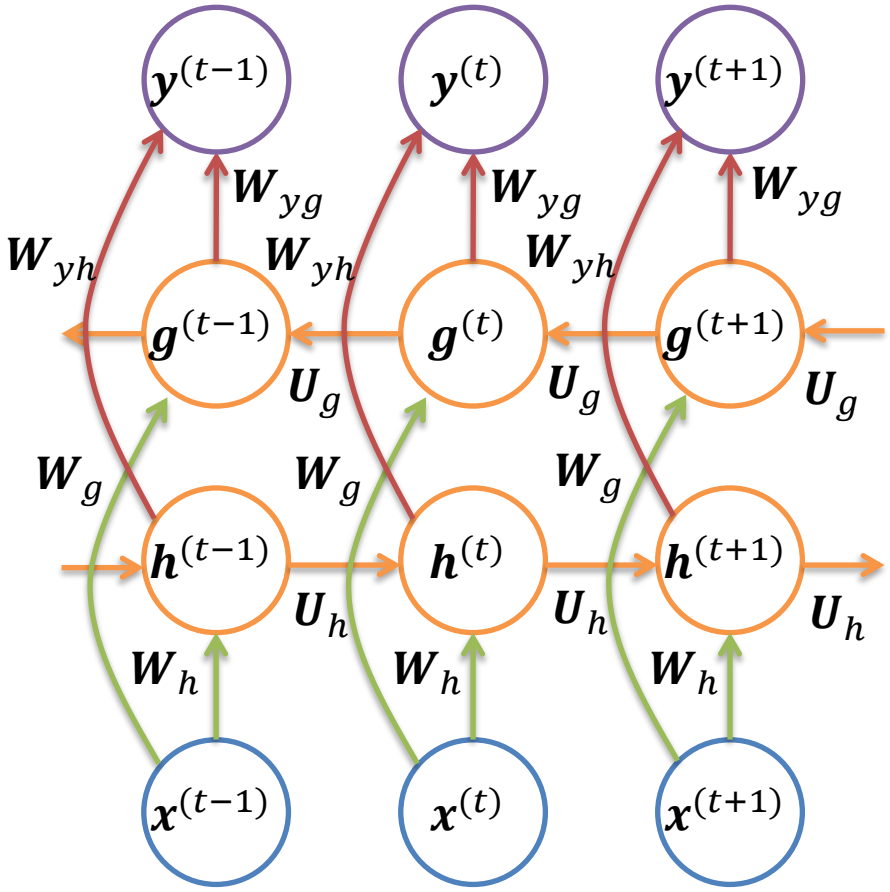
Bidirectional RNN



W_{yg}: connection weights from the sub-RNN g to output

One sub-RNN moves backward

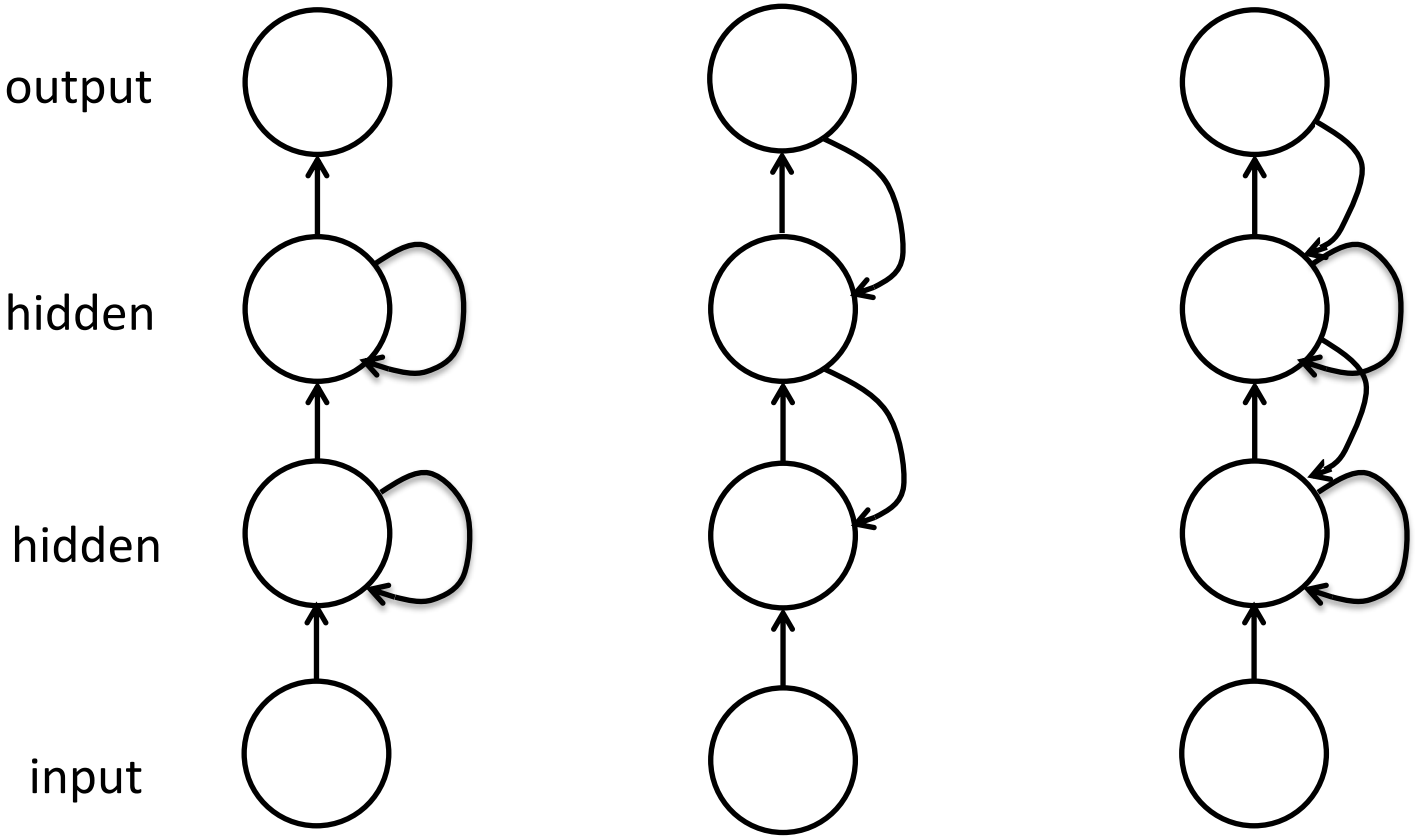
Bidirectional RNN



The output $\boldsymbol{y}^{(t)}$ recives input from both sub-RNNs via $\boldsymbol{W}_{yh}$ and $\boldsymbol{W}_{yg}$

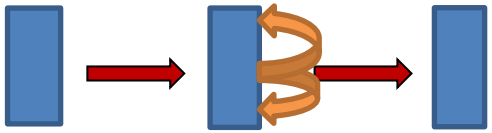
Deep RNNs

- Many ways to construct deep RNNs

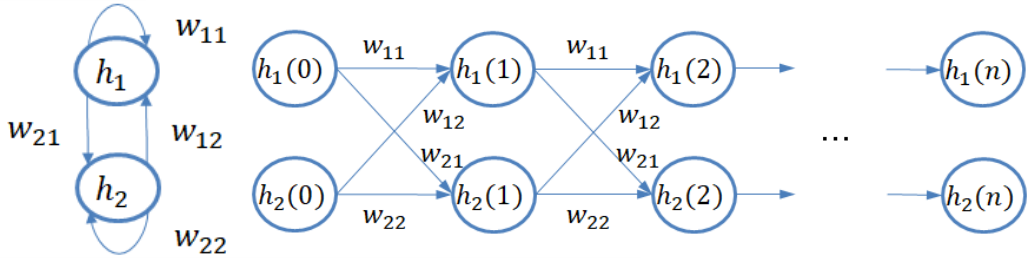


Summary of Part I

Elman network

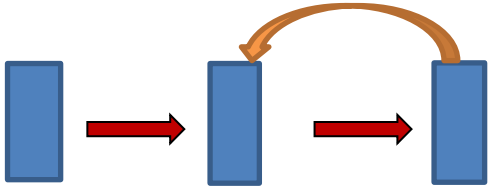


BPTT

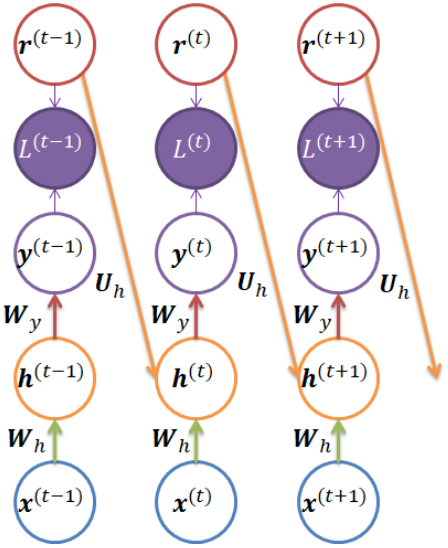


Different unfolding schemes

Jordan network



Teacher forcing



Bidirectional RNN

Deep RNN

单选题 1分

Setting

If you want your RNN’s prediction depends on both history and future, which model would you choose?

A Deep RNN

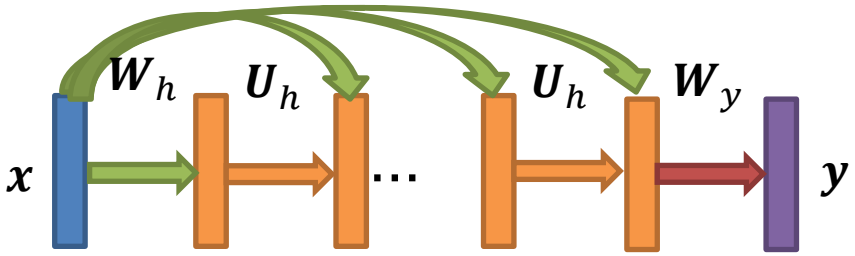
B Bidirectional RNN

Submit

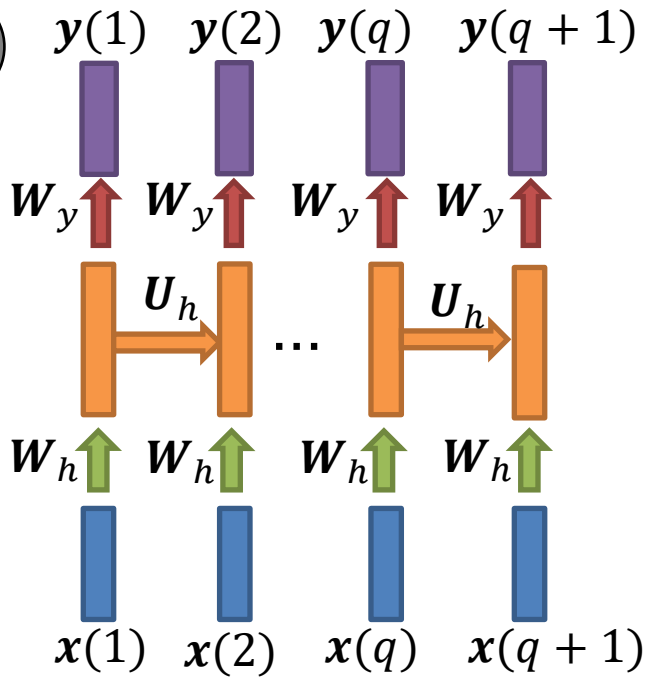
Setting

Consider using an RNN to do translation, which unfolding scheme best suits this task?

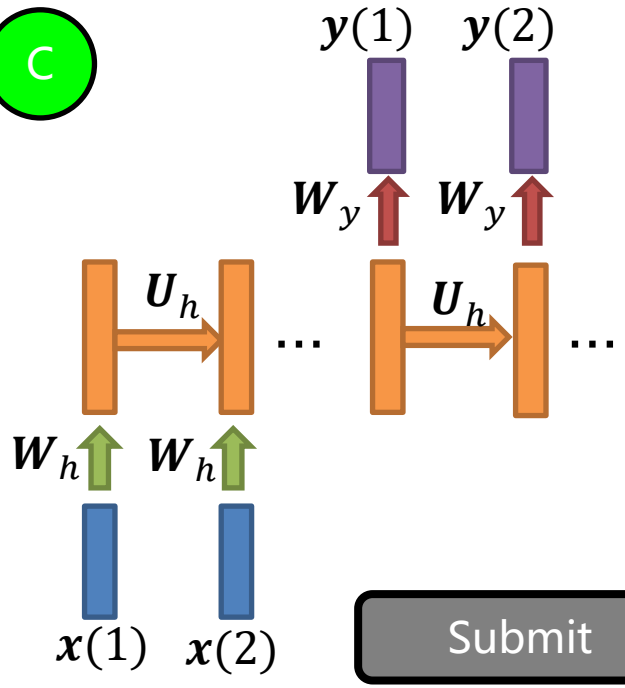
A



B



C

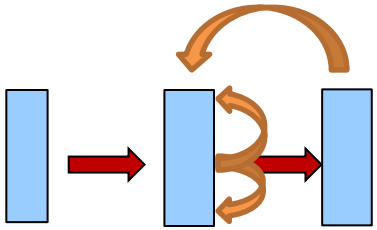


Submit

单选题 1分

Setting

Can we use teacher forcing to train an RNN with both hidden-to-hidden and output-to-hidden recurrent connections?



A Yes

B No

Submit

Outline

I. Simple RNNs
II. Gated RNNs
III. Applications to speech recognition
IV. Summary

Challenges to the Elman network

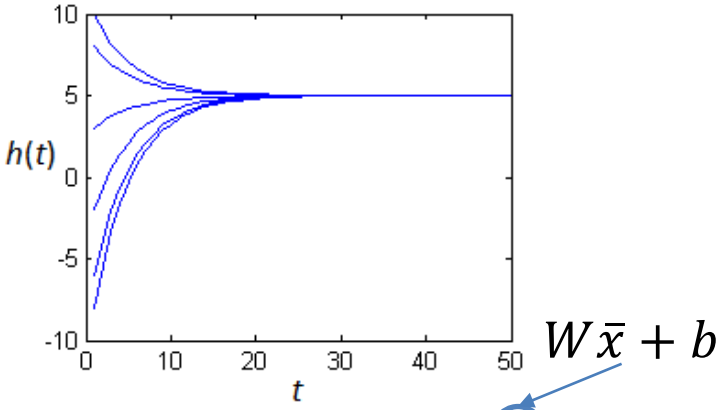
- Consider the 1D Elman network

$$h(t) = \sigma_h(Wx(t) + Uh(t-1) + b)$$

- Suppose σ_h is an identity mapping, $x(t)$ is a constant $\bar{x}$

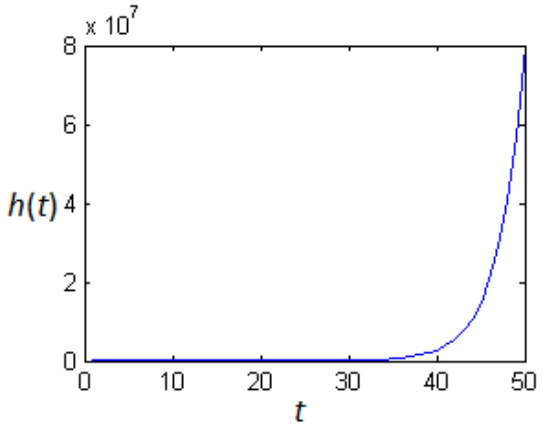
$$h(t) = Uh(t-1) + W\bar{x} + b$$

  - If $|U| > 1$, $h(t)$ will approach infinity
  - If $|U| < 1$, $h(t)$ will converge to a fixed point



$W\bar{x} + b$

$$h(t+1) = 0.8h(t) + 1$$



$$h(t+1) = 1.4h(t) + 1$$

Challenges to the Elman network

- Even for time-varying $x(t)$, when $|U| > 1$, $h(t)$ may also approach infinity

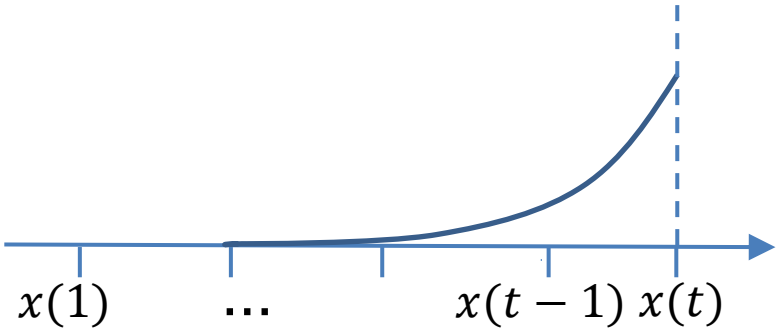
$$h(t) = Uh(t-1) + Wx(t) + b$$

  - Let $b = 0$. After t steps from zero

$$h(t) = U^{t-1}Wx(1) + U^{t-2}Wx(2) + \cdots + Wx(t) + U^t h(0) \quad \rightarrow \infty$$

- What if $|U| < 1$?

Contribution to $h(t)$



The contribution of $x(t-\tau)$ to $h(t)$ exponentially decays when $\tau > 0$ increases

$h(t)$ is mainly determined by recent input

The short-term memory is too short!

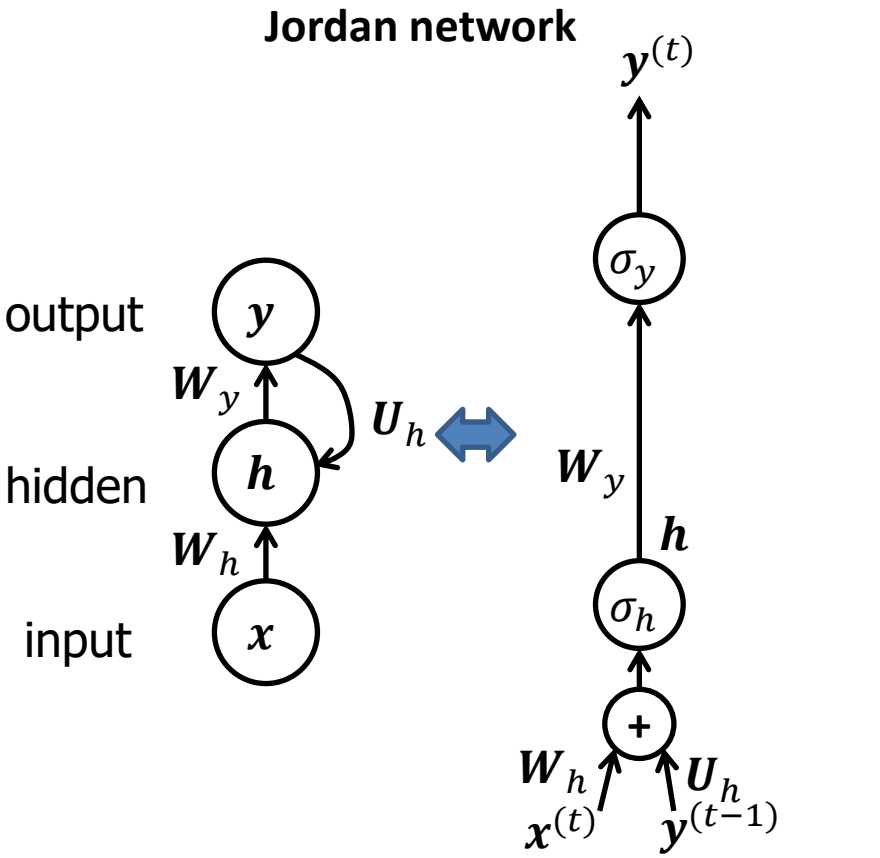
Remarks

- Similar arguments hold for multi-dimensional Elman network
  - Some assumptions on U are needed, e.g., U is symmetric and has eigenvalue-eigenvector decomposition
- Same conclusions can be drawn on the Jordan network
  - You can express $\boldsymbol{y}(t)$ as a function of $\boldsymbol{y}(0)$ and $\boldsymbol{x}(1), \ldots, \boldsymbol{x}(t)$

How to make the memory longer?

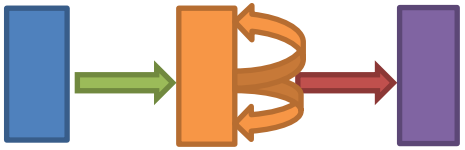
Long short-term memory (LSTM) cell

- A combination of the Jordan network and the Elman network
  - The output is connect to the input
  - A self-loop is used to capture the information about the past
- Redraw the Jordan network
  - Use circles to denote operations
  - Variables are indicated on arrows
  - Bias $\boldsymbol{b}$ is ignored

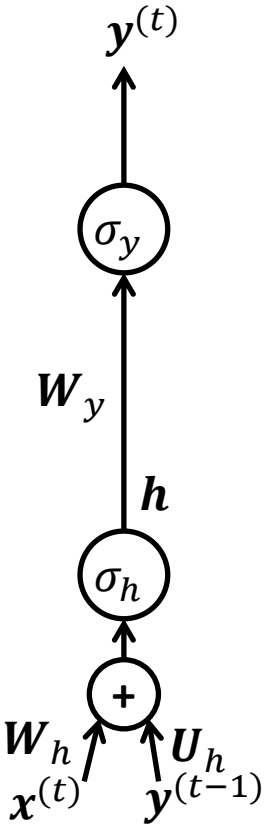


$$\boldsymbol{h}^{(t)} = \sigma_h\left(\boldsymbol{W}_h\boldsymbol{x}^{(t)} + \boldsymbol{U}_h\boldsymbol{y}^{(t-1)} + \boldsymbol{b}_h\right)$$
$$\boldsymbol{y}^{(t)} = \sigma_y\left(\boldsymbol{W}_y\boldsymbol{h}^{(t)} + \boldsymbol{b}_y\right)$$

Step 1: add a self-loop



Jordan network

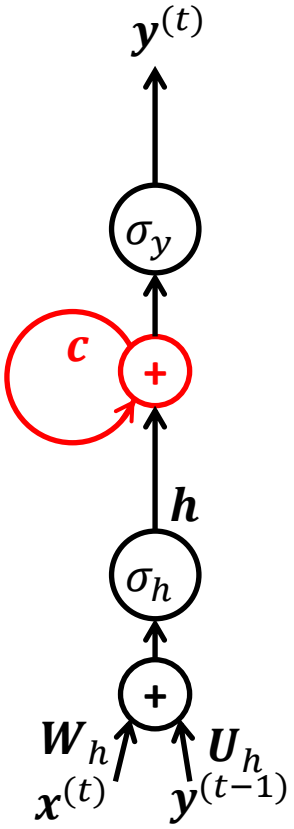


$$\boldsymbol{h}^{(t)} = \sigma_h\left(\boldsymbol{W}_h\boldsymbol{x}^{(t)} + \boldsymbol{U}_h\boldsymbol{y}^{(t-1)} + \boldsymbol{b}_h\right)$$
$$\boldsymbol{y}^{(t)} = \sigma_y(\boldsymbol{W}_y\boldsymbol{h}^{(t)} + \boldsymbol{b}_y)$$

$$\boldsymbol{h}^{(t)} = \sigma_h\left(\boldsymbol{W}_h\boldsymbol{x}^{(t)} + \boldsymbol{U}_h\boldsymbol{y}^{(t-1)} + \boldsymbol{b}_h\right)$$
$$\boldsymbol{c}^{(t)} = \boldsymbol{c}^{(t-1)} + \boldsymbol{h}^{(t)}$$
$$\boldsymbol{y}^{(t)} = \sigma_y(\boldsymbol{c}^{(t)})$$

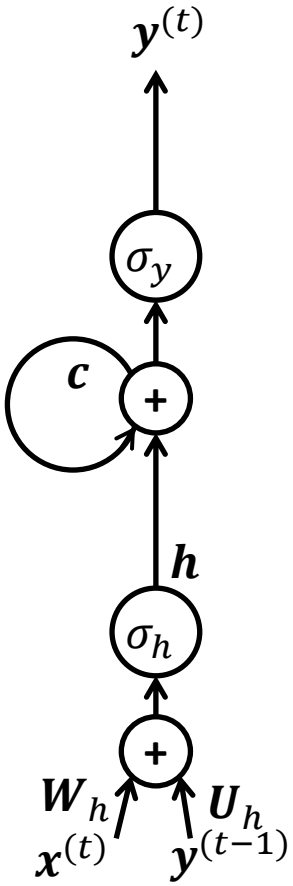
(We eliminate the linear transformation in the output)

- σ_h is either the logistic sigmoid function or tanh function
- σ_y is often tanh function



Step 2: add three gates

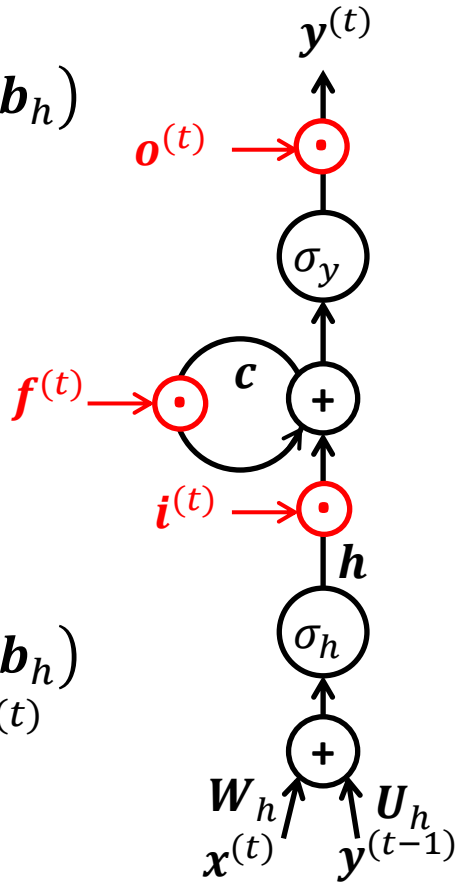
Gates are introduced to adaptively control the flow of information



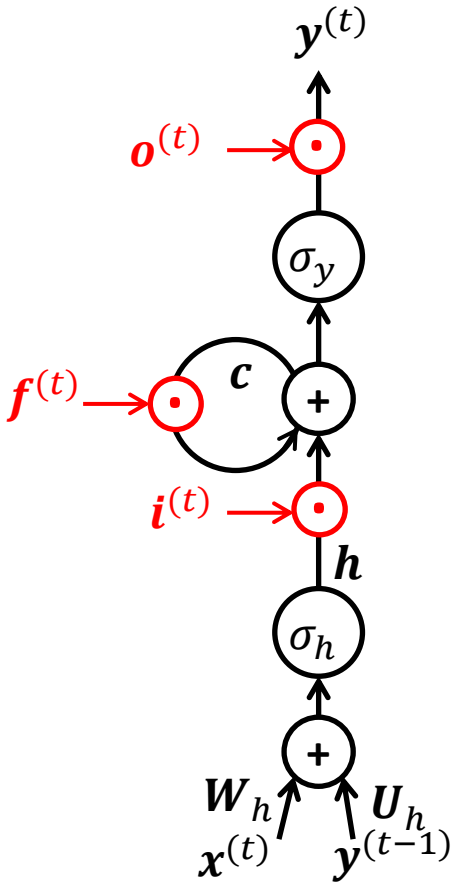
$$h^{(t)} = \sigma_h\left(W_h x^{(t)} + U_h y^{(t-1)} + b_h\right)$$
$$c^{(t)} = c^{(t-1)} + h^{(t)}$$
$$y^{(t)} = \sigma_y(c^{(t)})$$

Forget gate $f^{(t)}$, input gate $i^{(t)}$, output gate $o^{(t)}$: between (0,1)

$$h^{(t)} = \sigma_h\left(W_h x^{(t)} + U_h y^{(t-1)} + b_h\right)$$
$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot h^{(t)}$$
$$y^{(t)} = o^{(t)} \odot \sigma_y(c^{(t)})$$



What determine these gates?



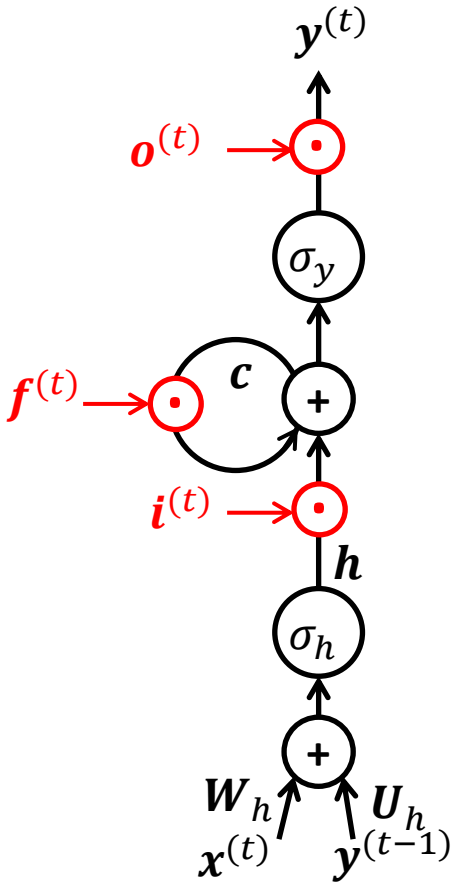
$$\boldsymbol{h}^{(t)} = \sigma_h\left(\boldsymbol{W}_h\boldsymbol{x}^{(t)} + \boldsymbol{U}_h\boldsymbol{y}^{(t-1)} + \boldsymbol{b}_h\right)$$
$$\boldsymbol{c}^{(t)} = \boldsymbol{f}^{(t)} \odot \boldsymbol{c}^{(t-1)} + \boldsymbol{i}^{(t)} \odot \boldsymbol{h}^{(t)}$$
$$\boldsymbol{y}^{(t)} = \boldsymbol{o}^{(t)} \odot \sigma_y(\boldsymbol{c}^{(t)})$$

- All of the gates are determined by the input $\boldsymbol{x}^{(t)}$ and $\boldsymbol{y}^{(t-1)}$

$$\boldsymbol{f}^{(t)} = \sigma\left(\boldsymbol{W}_f\boldsymbol{x}^{(t)} + \boldsymbol{U}_f\boldsymbol{y}^{(t-1)} + \boldsymbol{b}_f\right)$$
$$\boldsymbol{i}^{(t)} = \sigma\left(\boldsymbol{W}_i\boldsymbol{x}^{(t)} + \boldsymbol{U}_i\boldsymbol{y}^{(t-1)} + \boldsymbol{b}_i\right)$$
$$\boldsymbol{o}^{(t)} = \sigma\left(\boldsymbol{W}_o\boldsymbol{x}^{(t)} + \boldsymbol{U}_o\boldsymbol{y}^{(t-1)} + \boldsymbol{b}_o\right)$$

  where σ is the logistic sigmoid function
- Sometimes, they are also determined by $\boldsymbol{c}^{(t)}$ and $\boldsymbol{c}^{(t-1)}$: *peepholes*

Terminology



$$h^{(t)} = \sigma_h\left(W_h x^{(t)} + U_h y^{(t-1)} + b_h\right)$$
$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot h^{(t)}$$
$$y^{(t)} = o^{(t)} \odot \sigma_y(c^{(t)})$$

- $x^{(t)}$: input
- $y^{(t)}$: output
- $h^{(t)}$ and $c^{(t)}$: hidden states
- $f^{(t)}$, $i^{(t)}$ and $o^{(t)}$: gates

Note: sometimes, the output y is also called *hidden state of LSTM*, especially when LSTM is integrated into a larger system.

单选题 1分

Setting

$$\boldsymbol{h}^{(t)} = \sigma_h\left(\boldsymbol{W}_h\boldsymbol{x}^{(t)} + \boldsymbol{U}_h\boldsymbol{y}^{(t-1)} + \boldsymbol{b}_h\right)$$
$$\boldsymbol{c}^{(t)} = \boldsymbol{f}^{(t)} \odot \boldsymbol{c}^{(t-1)} + \boldsymbol{i}^{(t)} \odot \boldsymbol{h}^{(t)}$$
$$\boldsymbol{y}^{(t)} = \boldsymbol{o}^{(t)} \odot \sigma_y(\boldsymbol{c}^{(t)})$$

What' s the ideal case for keeping the memory $\boldsymbol{c}^{(n)}$ obtained at $t = n$ forever?

A $\boldsymbol{i}^{(t)} = 1$ and $\boldsymbol{o}^{(t)} = 1$ for $t \geq n + 1$

B $\boldsymbol{i}^{(t)} = 1$ and $\boldsymbol{f}^{(t)} = 0$ for $t \geq n + 1$

C $\boldsymbol{i}^{(t)} = 0$ and $\boldsymbol{f}^{(t)} = 1$ for $t \geq n + 1$

Submit

Advantage of LSTM

- The gates enable the model to keep the memory for a longer time than simple RNNs

  "Long short-term memory network"

- Gates have been widely used in deep learning models, not only in RNNs but also CNNs
  - Highway Network (Srivastava et al., ICML 2015 Deep Learning workshop)
  - SEnet (Hu et al., CVPR 2018)
  - SKnet (Li et al., CVPR 2019)

  It is used in a more broad sense: attention

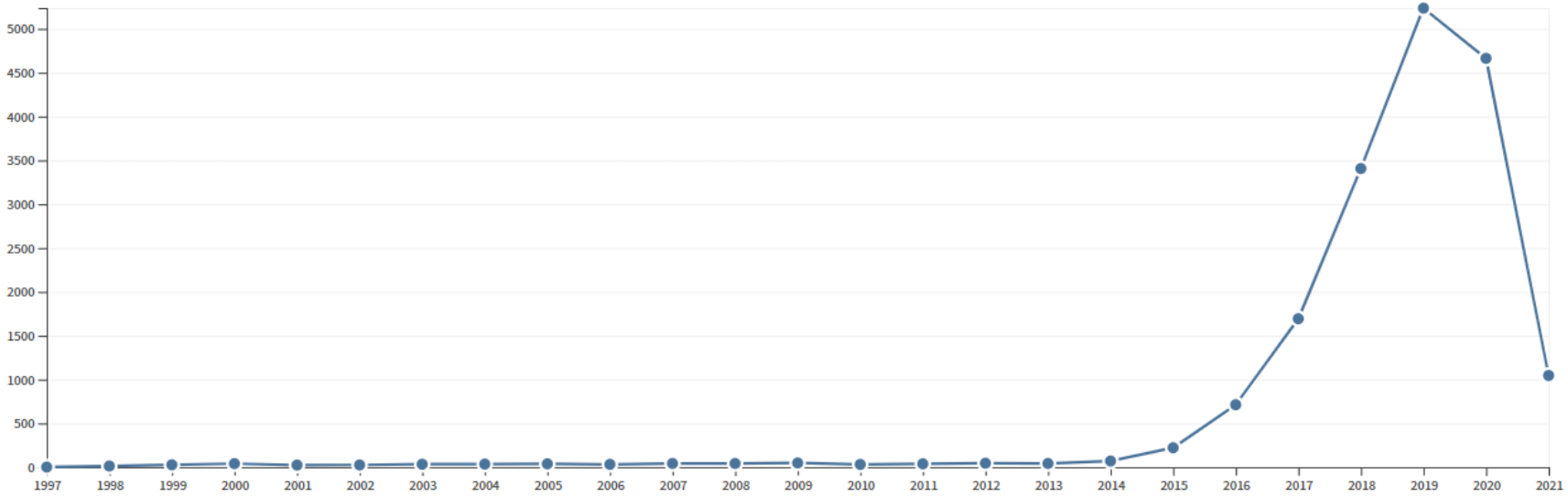
A good work needs time to be recognized

Hochreiter, S, Schmidhuber, J, Long short-term memory, Neural Computation, vol 9, no. 8, pp. 1735-1780, 1997

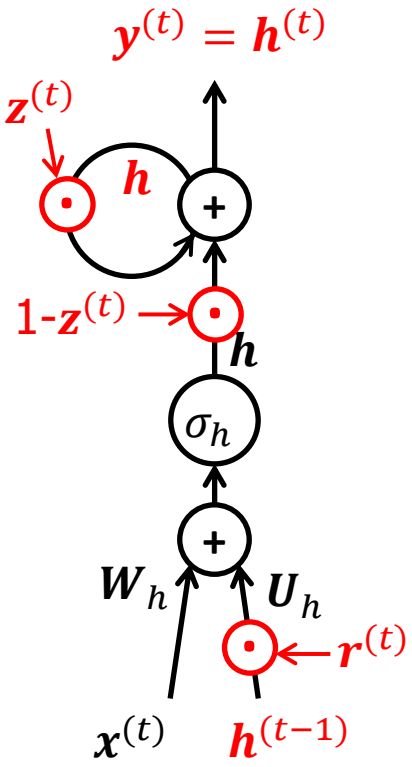


Jürgen Schmidhuber

Citation report by Web of Science (April 7, 2021)



Gated recurrent unit (GRU)



- Dynamic equations

$$\boldsymbol{h}^{(t)} = \boldsymbol{z}^{(t)} \odot \boldsymbol{h}^{(t-1)} + \left(1 - \boldsymbol{z}^{(t)}\right) \odot \tilde{\boldsymbol{h}}^{(t)}$$

$$\tilde{\boldsymbol{h}}^{(t)} = \sigma_h\left(\boldsymbol{W}_h \boldsymbol{x}^{(t)} + \boldsymbol{U}_h\left(\boldsymbol{r}^{(t)} \odot \boldsymbol{h}^{(t-1)}\right) + \boldsymbol{b}_h\right)$$

where σ_h is either the logistic sigmoid function or tanh function

- The gates

$$\boldsymbol{z}^{(t)} = \sigma\left(\boldsymbol{W}_z \boldsymbol{x}^{(t)} + \boldsymbol{U}_z \boldsymbol{h}^{(t-1)} + \boldsymbol{b}_z\right)$$

$$\boldsymbol{r}^{(t)} = \sigma\left(\boldsymbol{W}_r \boldsymbol{x}^{(t)} + \boldsymbol{U}_r \boldsymbol{h}^{(t-1)} + \boldsymbol{b}_r\right)$$

σ is the logistic sigmoid function

Watch the video material

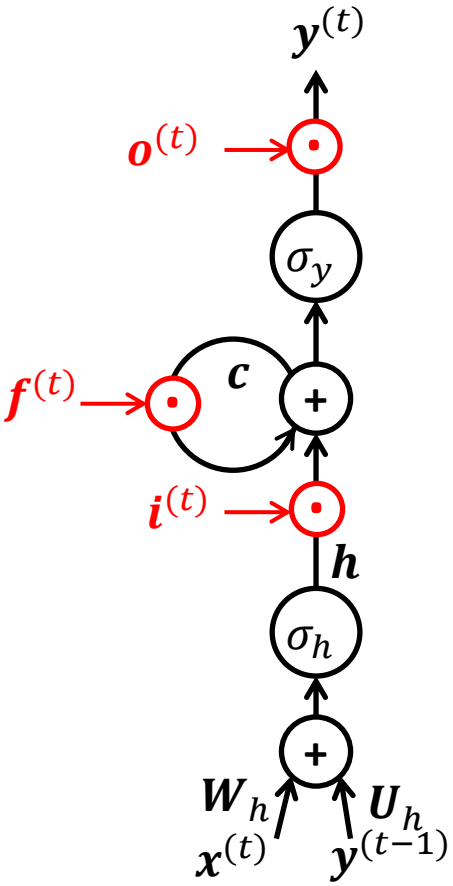
Which is better?

- LSTM and GRU perform similarly on many tasks
- There are many other variants of LSTM and GRU, but none of them would clearly beat these two across a wide range of tasks (Greff et al., TNNLS 2017)

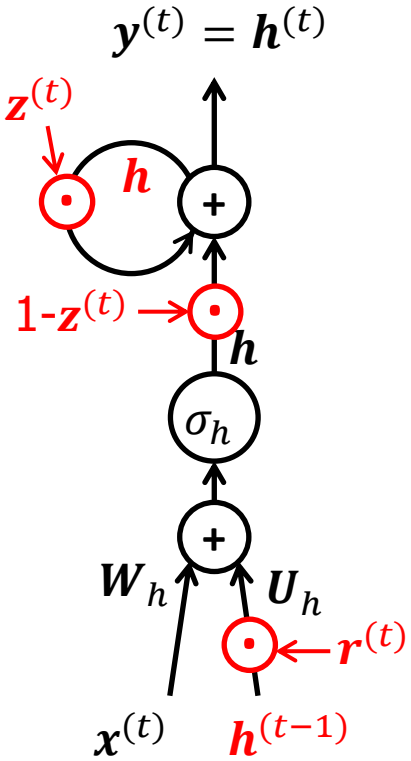
Greff, Srivastava, Koutník, et al., ”LSTM: A Search Space Odyssey,” IEEE TNNLS 2017

Summary of Part II

LSTM



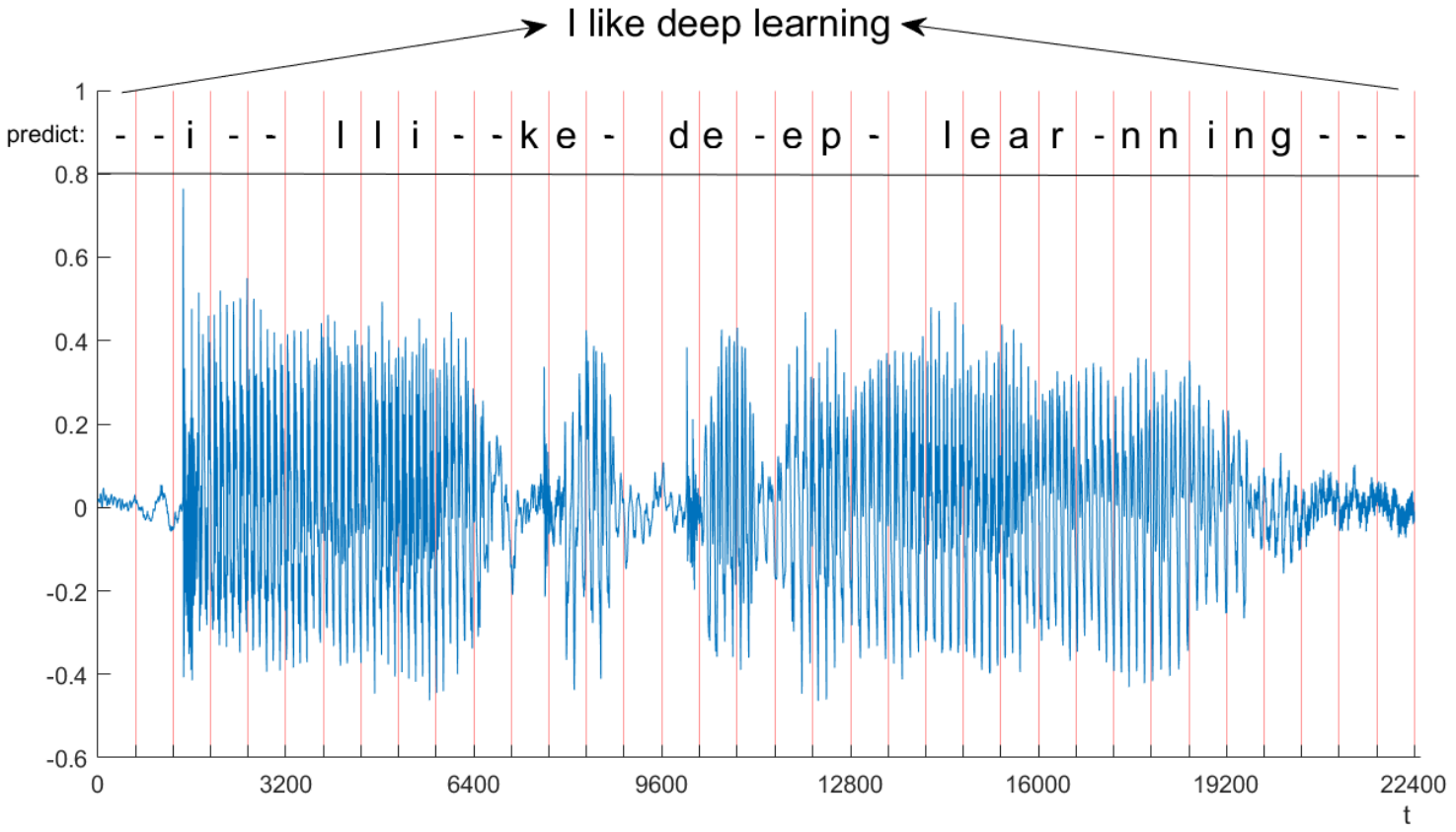
GRU



Outline

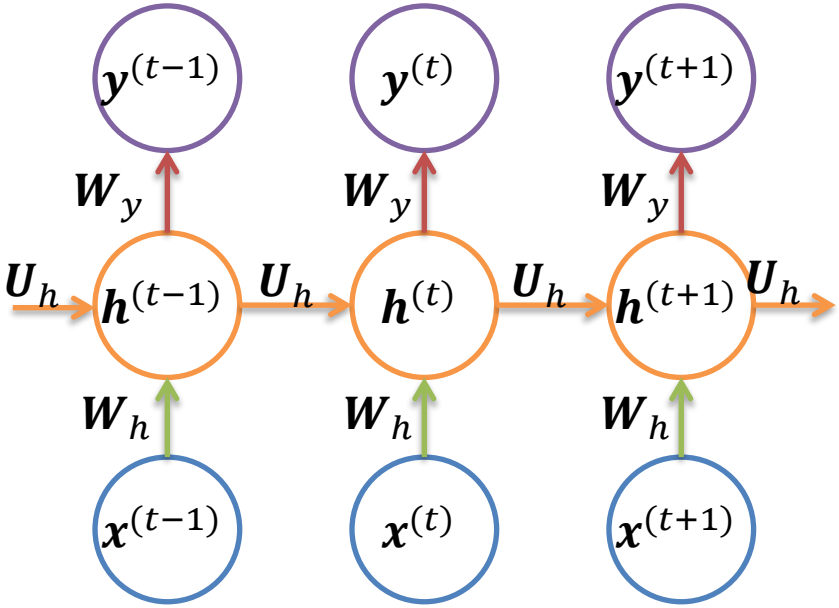
I. Simple RNNs
II. Gated RNNs
III. Applications to speech recognition
IV. Summary

Speech recognition



RNN setting

Reference $\boldsymbol{r}^{(t)}$ at each time step



- Suppose there is no act fun

$$\boldsymbol{y}^{(t)} = \boldsymbol{W}_y \boldsymbol{h}^{(t)} + \boldsymbol{b}_y$$

- At every time step, use a softmax fun to predict an output, e.g., a phoneme or a blank
  - There are $K+1$ classes at time t, where K is the number of phonemes, usually < 100

$$P\left(\mathrm{r}_k^{(t)} = 1 | \boldsymbol{h}^{(t)}\right) = \frac{\exp\left(y_k^{(t)}\right)}{\sum_{k=1}^{K+1} \exp\left(y_k^{(t)}\right)}$$

A random variable, not reference value

Objective function

- Maximize the prob of reference class at all time steps

$$\max_{\boldsymbol{\theta}} \sum_{t=1}^{T} \ln P\left(\mathrm{r}_k^{(t)} = r_k^{(t)} | \boldsymbol{h}^{(t)}\right)$$

Reference value which is 1

- This is equivalent to minimizing the cross-entropy error
  - The cross-entropy error at time t

$$-\sum_{k=1}^{K+1} r_k^{(t)} \ln p\left(\mathrm{r}_k^{(t)} = 1 \middle| \boldsymbol{h}^{(t)}\right) = -\ln p\left(\mathrm{r}_k^{(t)} = 1 \middle| \boldsymbol{h}^{(t)}\right)$$

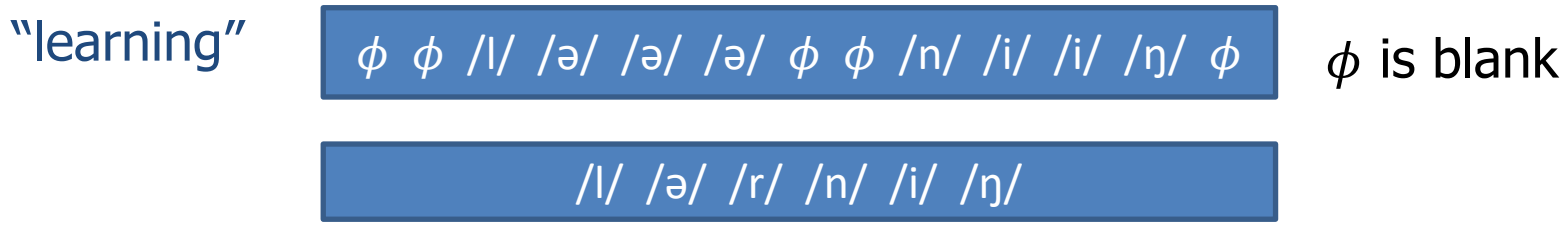
    where k satisfies $r_k^{(t)} = 1$ because other elements of $\boldsymbol{r}^{(t)}$ are zeros

  - Sum the cross-entropy error over time

$$-\sum_{t=1}^{T} \ln p\left(\mathrm{r}_k^{(t)} = 1 \middle| \boldsymbol{h}^{(t)}\right)$$

Objective function

- Optimizing the objective function in the previous slide will result in T outputs

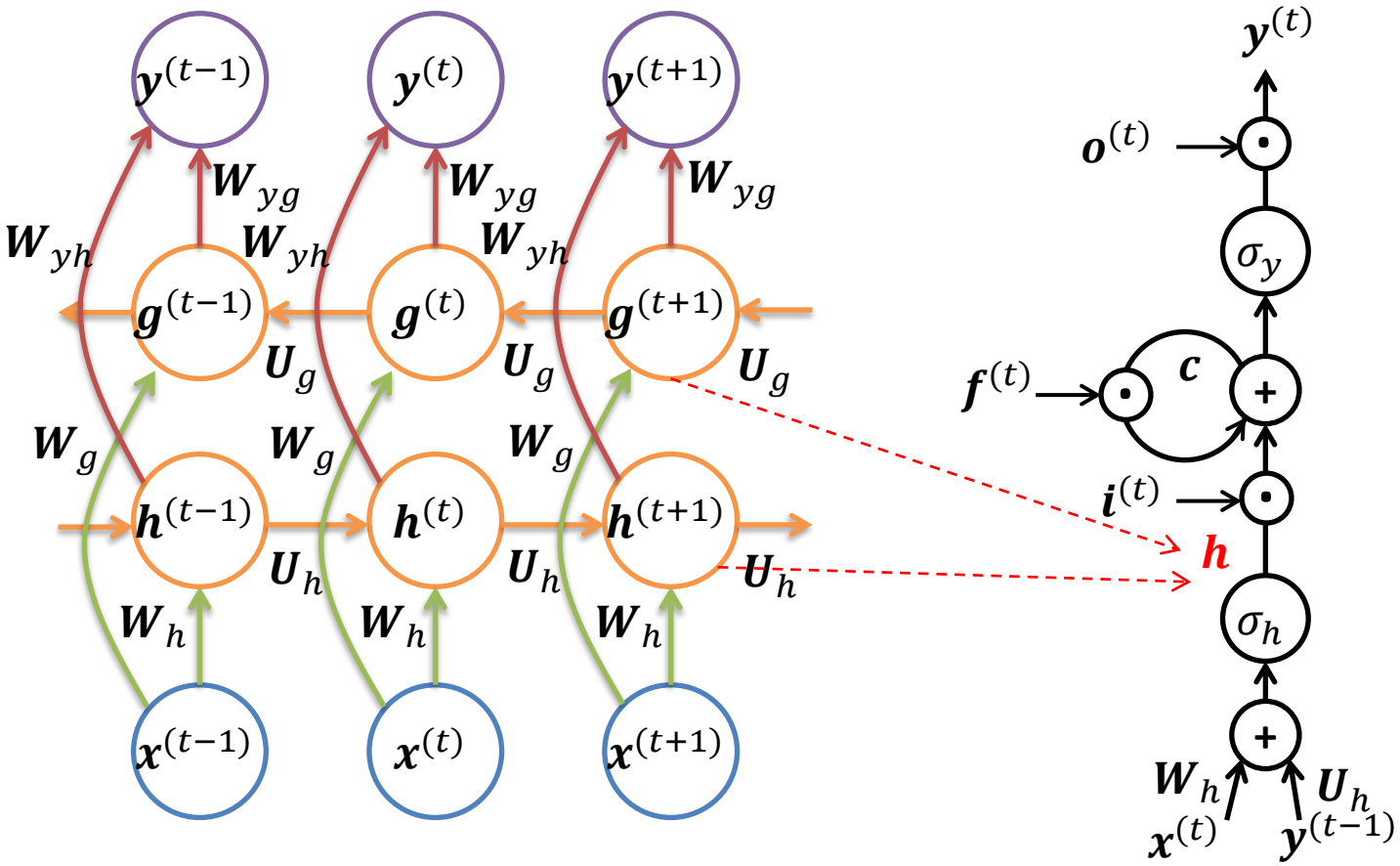


- But the reference sequence may be shorter or longer than it
- How to measure the difference and minimize the difference?
  - "edit-distance" is introduced: the minimum number of insertions, substitutions and deletions required to change seq p into seq q
  - A method called "Connectionist Temporal Classification (CTC)" is usually used (Graves et al., 2006)

Use bidirectional LSTM

Graves et al., 2013

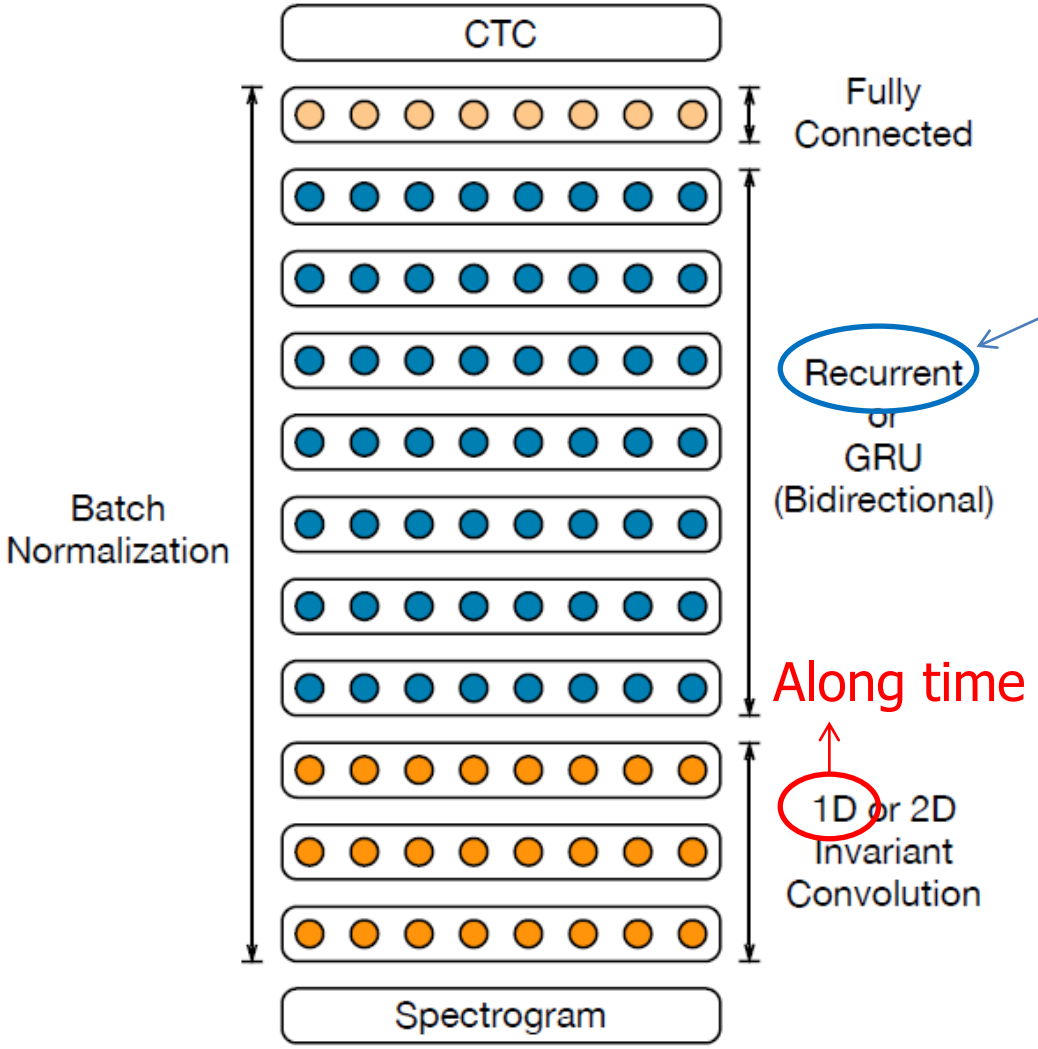
Both RNNs are LSTMs



Define the obj fun the same as before based on: $\boldsymbol{y}^{(t)} = \boldsymbol{W}_{yh}\boldsymbol{h}^{(t)} + \boldsymbol{W}_{yg}\boldsymbol{g}^{(t)} + \boldsymbol{b}_y$

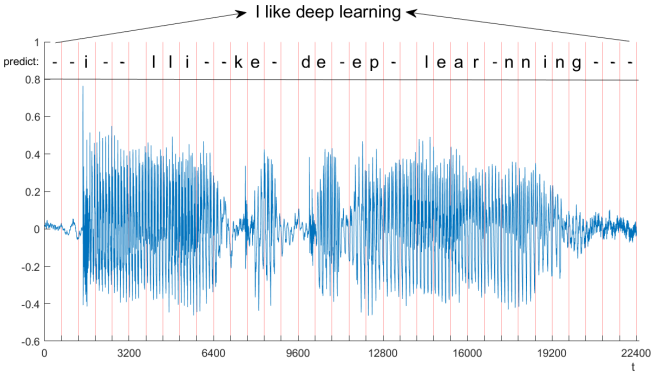
Deep RNN – Deep Speech 2 by Baidu

Amodei et al., 2015

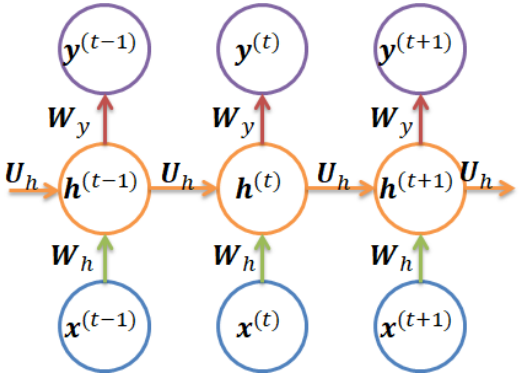


Variants were explored by varying the number of convolutional layers from 1 to 3 and the number of recurrent or GRU layers from 1 to 7

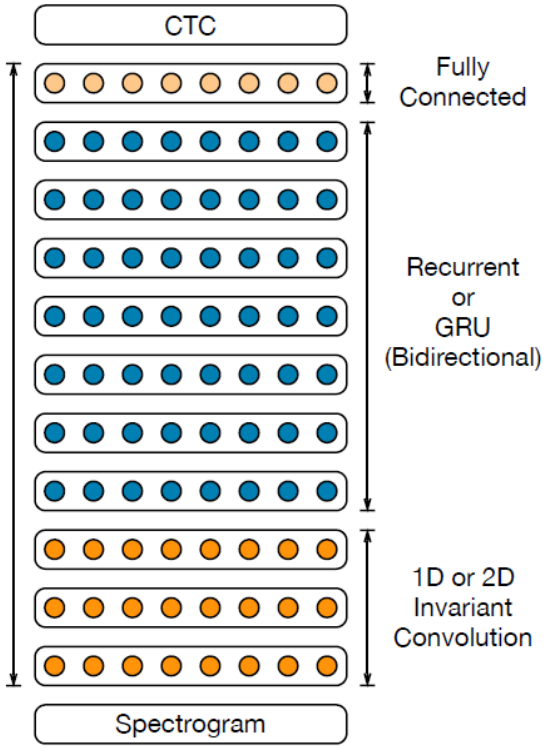
Summary of Part III



Elman network



Deep Speech 2



Difficulty: difference between sequences

Outline

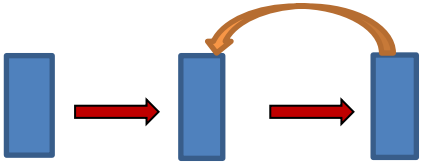
I. Simple RNNs
II. Gated RNNs
III. Applications to speech recognition
IV. Summary

Summary of this lecture

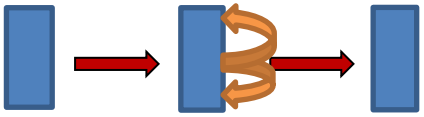
Knowledge

I. Simple RNNs

Jordan network



Elman network



Bidirectional RNN

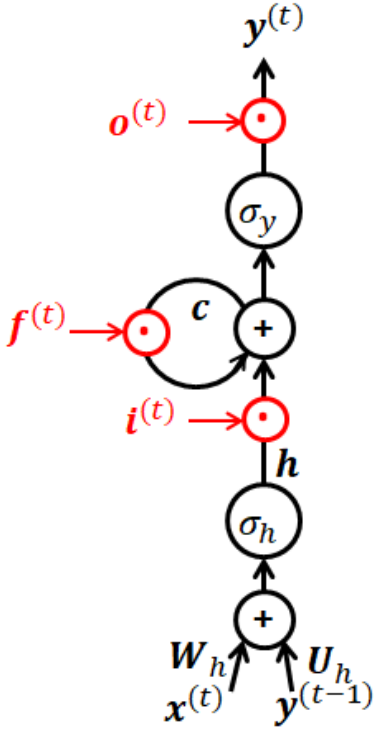
Deep RNN

BPTT

Teacher forcing

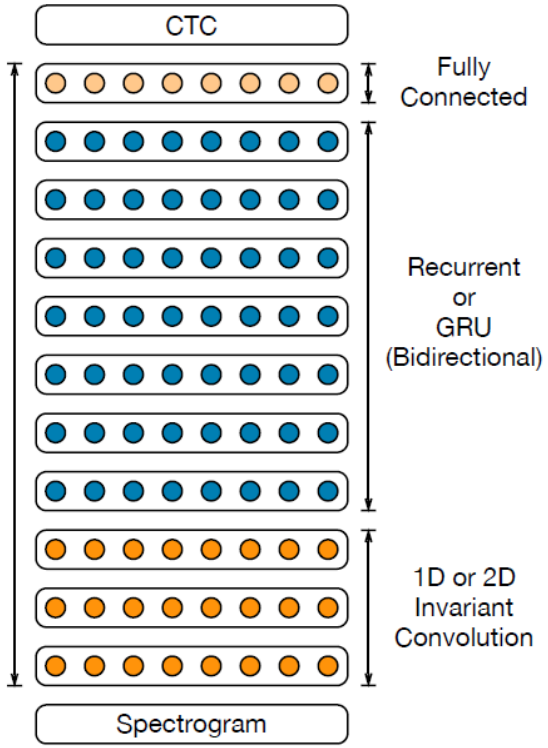
II. Gated RNNs

LSTM



III. Speech recog.

Deep Speech 2



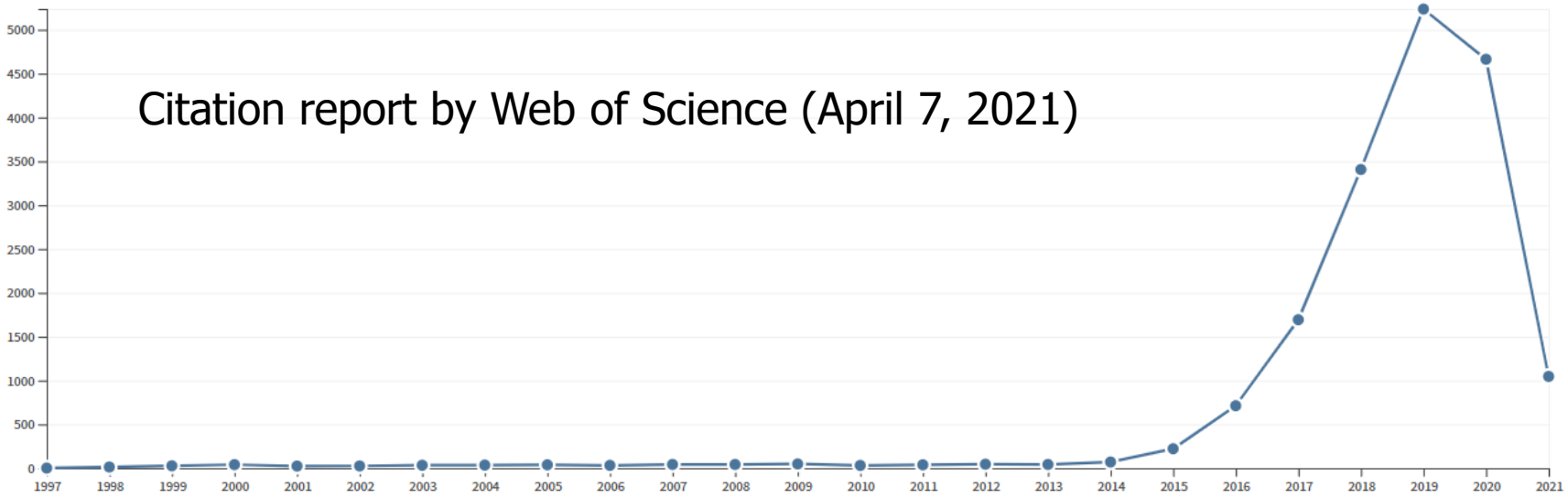
Summary of this lecture

Value

Hochreiter, S, Schmidhuber, J, Long short-term memory, Neural Computation, vol 9, no. 8, pp. 1735-1780, 1997



Jürgen Schmidhuber



A good work needs time to be recognized!

Recommended reading

- Goodfellow, Bengio and Courville, 2016
  Deep Learning, MIT Press, Chapters 10
- Understanding LSTM networks
  http://colah.github.io/posts/2015-08-Understanding-LSTMs/

Video material

- Watch the supplementary video uploaded to WebLearning
  - Gated recurrent Unit (GRU)